

Capítulo 4

Consultas SQL simples: selecionando e projetando

Uma consulta SQL é composta por três cláusulas principais¹:

```
select <lista alvo>  
from <fontes de registros>  
where <critérios de filtragem>
```

O papel de cada cláusula é descrever a consulta no que se refere a

- o quê ?** A cláusula **select** serve para especificar quais e como são as colunas do resultado final;
- de onde ?** A cláusula **from** serve para indicar as fontes dos registros que serão considerados e combinados para formar o resultado final;
- quando ?** Potencialmente, todos os registros referenciados na cláusula *from* participam do resultado, de acordo com a especificação da cláusula *select*. A cláusula **where** é um filtro que determina quais os registros que efetivamente devem ser considerados. A cláusula *where* é opcional.

O resultado de uma consulta SQL é construído na forma de uma tabela, com zero ou mais linhas, onde cada coluna recebe um nome.

Vejamos um breve exemplo, que produz a lista das matrículas e nomes dos alunos do sexo feminino.

```
select matricula, nome  
from aluno  
where sexo = 'F'
```



matricula	nome
1002	Maria Rita Colatti
1005	Barbara Carlito
1010	Maria Lucia Silva

o quê ?

- A cláusula **select** especifica que o resultado deve conter as colunas *matricula* e *nome*.

de onde ?

- A cláusula **from** indica que a fonte de registros é a tabela **ALUNO**. Por enquanto, vamos considerar que as fontes de registros são sempre tabelas do banco de dados. Nos capítulos se-

¹ A cláusula *where* é opcional. A cláusula *from* é opcional no MySQL, Oracle e SQL Server.

guintes, veremos que é possível ter outras construções para indicar a partir de onde o resultado é construído.

quando ?

- A cláusula **where** indica que somente os alunos do sexo feminino devem aparecer no resultado, o que é especificado através de uma expressão lógica. Todo registro da tabela ALUNO, para o qual a expressão lógica é verdadeira, é utilizado na construção do resultado. No exemplo, essa condição é verdadeira sempre que o atributo *sexo* tiver o valor “F”. A omissão da cláusula *where* equivale a dizer que não há restrições ou que todos os registros de entrada participam do resultado.

O exemplo a seguir mostra uma consulta SQL bastante simples².

select *
from aluno



matricula	nome	sexo	codcurso	nascimento
1001	Ricardo Biondi	M	DIR	21/02/1980
1002	Maria Rita Colatti	F	INF	10/11/1978
1004	Oscarito Vianna	M	DIR	14/08/1979
1005	Barbara Carlito	F	JOR	29/10/1979
1007	Carlos Maradona	M	DIR	30/06/1977
1008	Sacadura Miranda	M	INF	12/12/1981
1010	Maria Lucia Silva	F	JOR	10/08/1975

O símbolo * denota *todos os atributos* e é uma forma simplificada de especificar as colunas que compõem o resultado. Em vez de nomear as colunas, uma a uma, basta usar o símbolo referido e *todas* as colunas são consideradas.

Os alunos do curso de Direito correspondem aos registros da tabela ALUNO que têm o valor “DIR” na coluna *codcurso*. O próximo exemplo produz uma lista com matrículas e nomes dos alunos de Direito, e equivale a uma operação de seleção seguida de uma operação de projeção.

select matricula, nome
from aluno
where CodCurso = 'DIR'



matricula	nome
1001	Ricardo Biondi
1004	Oscarito Vianna
1007	Carlos Maradona

O critério de filtragem é expresso através da cláusula *where* no comando SQL. De acordo com os dados da tabela ALUNO, três registros passam no teste. O critério de seleção pode ser tão complexo quanto se queira. No exemplo a seguir, o critério envolve a conjunção de dois fatores.

select matricula, nome
from aluno
where CodCurso = 'INF' **and** Sexo = 'F'



matricula	nome
1002	Maria Rita Collati

Somente os registros dos alunos do curso de Informática (“INF”) que são do sexo feminino (“F”) devem ser considerados no resultado final. No nosso banco de dados, somente uma aluna preenche esses requisitos.

² Esta é uma das formas mais abreviadas de consulta SQL. Nas implementações onde a cláusula *from* é opcional (Access, MySQL, SQL Server), é até possível escrever uma consulta como **select** 1, que tem como resultado uma tabela com apenas uma linha com apenas uma coluna cujo valor é 1.

4.1 ANATOMIA DE UMA CONSULTA SQL SIMPLES

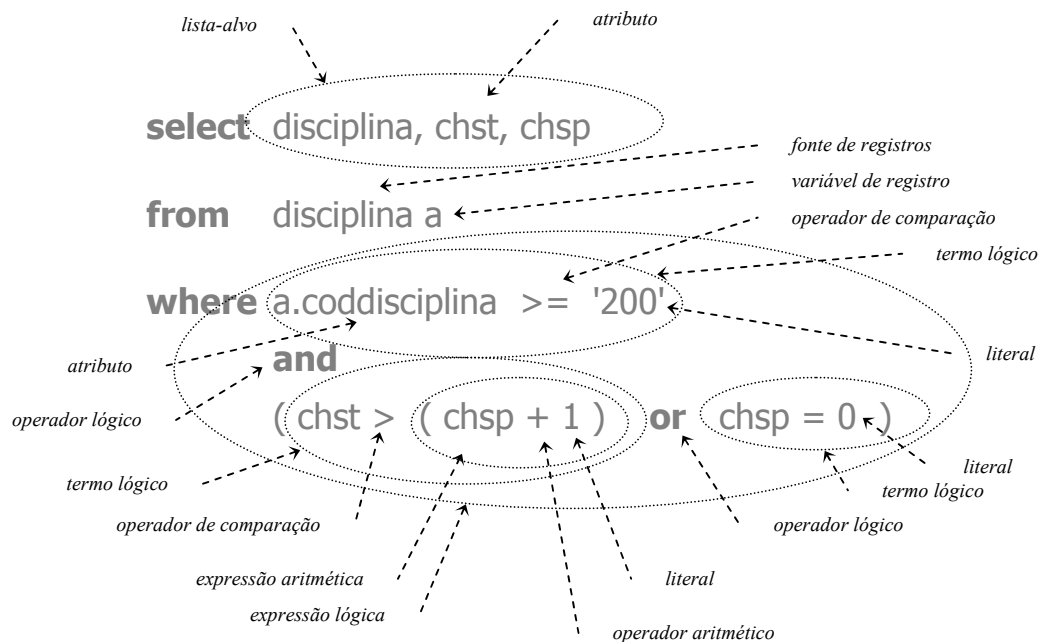
Neste ponto, é importante conhecer a natureza sintática dos principais elementos constituintes das consultas e comandos SQL. Considere a consulta a seguir.

```
select disciplina, chst, chsp
from disciplina a
where coddisciplina >= 200
and (chst > (chsp + 1) or chsp = 0 )
```

O resultado produzido contém os nomes e as cargas horárias das disciplinas que têm código igual ou superior a 200 e carga horária teórica com mais de uma hora a mais que a carga prática ou, alternativamente, que tenham a carga prática igual a zero.

disciplina	chst	chsp
Banco de dados	4	2
Sociologia	3	1
Português	4	0

A mesma consulta é reproduzida abaixo, num esquema que identifica a natureza de seus elementos sintáticos.



A primeira linha da consulta (cláusula *select*) contém a *lista-alvo*, que determina as colunas que devem aparecer no resultado. Essa *lista-alvo* é formada pelos valores dos atributos *disciplina*, *CHST*, *CHSP*.

Na segunda linha (cláusula *from*) pode-se observar um exemplo de *fonte de registros* e de uma *variável de registros*, respectivamente denominadas *disciplina* e *a*.

As demais linhas contém vários tipos de elementos: *atributos*, *termos*, *operadores lógicos* e *operadores aritméticos*. Note como os termos lógicos são combinados pelos operadores lógicos para formar expressões lógicas. Similarmente, atributos e literais podem ser combinados por operadores aritméticos na formação de expressões aritméticas.

O termo genérico *expressão* engloba:

- Atributos

- Literais
- Expressões aritméticas

Assim sendo, pode-se dizer que, na consulta apresentada, as construções

```
chsp
chsp + 2
'200'
( chsp + 2 )
```

são expressões.

4.2 LITERAIS, EXPRESSÕES ARITMÉTICAS E EXPRESSÕES LÓGICAS

Há vários tipos de literais que podem aparecer num comando SQL, designando números, cadeias de caracteres, datas, além de literais para valores booleanos e valores nulos.

Esta seção apresenta os aspectos genéricos que envolvem a escrita de literais. Ao final deste capítulo, as seções 4.15 até 4.18 apresentam as peculiaridades de cada implementação SQL neste quesito.

Dominar a codificação de literais é importante na utilização do SQL e o leitor deve estar atento às nuances que envolvem tal codificação, especialmente nos aspectos referentes a datas.

LITERAIS

Literais são constantes que denotam valores numéricos, reais ou inteiros, datas, *strings* de caracteres e valores *booleanos* (*true* ou *false*). Embora cada implementação SQL suporte seus próprios formatos, existe um conjunto de diretrizes comuns para a escrita de literais, descritas a seguir.

Números

Números inteiros não têm ponto decimal e podem ser precedidos de sinal. Exemplos são

```
123
0
-15632
+200
```

Números reais têm ponto decimal e podem ser precedidos de sinal. Podem também ser escritos na notação científica, que consiste de um número decimal (mantissa) seguido da letra E (maiúscula ou minúscula) seguido de um número inteiro com ou sem sinal (expoente). O valor final do número assim expresso é o produto da mantissa vezes 10 elevado ao referido expoente. Exemplos são

```
123.01
0.0000
-15.632
+2,345e5
-5E10
```

Note que os dois últimos valores equivalem a 234.500 e -50.000.000.000. Note que nem sempre um número real é armazenado no computador exatamente como escrito, haja vista que são manipulados como números de ponto flutuante.

∴



Embora em algumas implementações SQL a escolha do símbolo decimal nas configurações regionais do sistema operacional possa estender-se às interfaces, nos *scripts* SQL,

em geral, esse efeito não se verifica. Assim, mesmo quando a vírgula tiver sido escolhida como ponto decimal, os números reais nos comandos SQL devem continuar sendo escritos com ponto, a menos que sejam expressamente modificados.

Strings

Strings de caracteres devem ser escritas entre aspas, como nos exemplos a seguir.

'ABC'	ABC
" cDe "	cDE
` "ABC"'	"ABC"

Dependendo das configurações usados pelo gerenciador do banco de dados, podem ser utilizadas aspas simples ou duplas, sendo o primeiro tipo o mais comum e o mais recomendado.

Quando é preciso representar uma *string* que contenha aspas em seu interior, basta repetir as aspas em sequência. *n* aspas seguidas significam *n-1* aspas na *string* interpretada. Assim, por exemplo, valem as seguintes codificações:

<i>Uma string digitada como...</i>	<i>é interpretada como...</i>
'D'Ávila'	D'Ávila
''ABC''	'ABC'

Nota: neste exemplo, excepcionalmente, as aspas simples são escritas como ` e `', de modo a facilitar sua visualização.

Algumas implementações aceitam aspas simples ou duplas, dependendo das opções de controle vigentes. Veja mais detalhes no final deste capítulo.

Datas e horários

Literais que expressam datas e horários apresentam a maior diversidade de formas válidas dentre as diversas implementações. Infelizmente, não há uma forma que seja simultaneamente suportada pelas implementações analisadas neste livro.

No SQL do Access, uma data é escrita entre dois caracteres # (jogo da velha), com o mês, o dia e o ano separados por barras, nesta ordem; no Oracle e no SQL Server, as datas podem ser escritas como *strings* de caracteres, entre aspas simples (e às vezes duplas), com os componentes separados por barras, numa ordem que pode ser escolhida; no MySQL, datas são escritas com ou sem aspas, usando qualquer tipo de separador entre os componentes (ou mesmo nenhum), mas sempre na ordem ano, mês, dia. Ou seja, não há uma codificação de data que seja aceita pelas quatro implementações.

Veja alguns exemplos de datas válidas:

<i>Implementação</i>	<i>Formas válidas de codificação para a data 31/12/2010</i>
Access	#12/31/2005# CDate ('31/12/2010') CDate ('12/31/2010')
MySQL	'2010/12/31' 20101231 '2010.12.31' '2010%12%31'
Oracle	'31/12/2010' '12/31/2010'
SQL Server	'31/12/2010' '12/31/2010'

Quanto ao registro de tempo, o padrão é bem mais uniforme. Exemplos válidos para a codificação de data e horário são mostrados a seguir.

<i>Implementação</i>	<i>Formas válidas de literais para 13h 45m de 31/12/2005</i>
Access	#12/31/2010 13:45# CDate ('31/12/2010 13:45') CDate ('12/31/2010 13:45')
MySQL	'2010/12/31 13:45:00' 201012311345 '2010.12.31 13.45' '2010%12%31%13*45*00'
Oracle	'31/12/2010 13:45' '12/31/2010 13:45'
SQL Server	'31/12/2010 13:45' '12/31/2010 13:45'

Na tabela acima, algumas das variações na ordem dos componentes dependem das configurações regionais vigentes (especialmente no Windows).

Constantes

Usualmente, as constantes **null**, **true** e **false** podem ser empregadas no SQL, significando, respectivamente, o valor nulo e os valores lógicos *verdadeiro* e *falso*.

Hexadecimais

Literais com caracteres hexadecimais podem ser utilizados no Access, MySQL e SQL Server. Um literal hexadecimal tem uma sintaxe básica que consiste numa string de caracteres hexadecimais, com ou sem aspas, precedida por 0x (zero e a letra x). Como cada implementação apresenta características peculiares no tratamento desses literais, os mesmos serão ilustrados diretamente nas seções pertinentes ao final deste capítulo.

EXPRESSÕES ARITMÉTICAS

Expressões aritméticas são formadas a partir da combinação de literais e atributos com os operadores aritméticos. Os operadores aritméticos comuns às quatro implementações SQL abordadas neste livro são:

<i>Operador aritmético</i>	<i>Operação</i>
+	soma
-	subtração
*	multiplicação
/	divisão

O resultado de uma expressão aritmética é sempre um número ou, eventualmente, o valor nulo.



As implementações SQL oferecem operadores aritméticos adicionais, analisados separadamente ao final deste capítulo. Entretanto, é sempre possível expressar esses operadores adicionais a partir do conjunto básico.

EXPRESSÕES LÓGICAS

A cláusula *where* é constituída por uma expressão lógica, que é formada por um ou mais termos lógicos combinados através de operadores lógicos.

TERMOS LÓGICOS

Exemplos de termos lógicos são

```
codcurso = 'INF'
chst + chsp > 4
a.matricula = i.matricula
x in (1,2,3,7,8,10)
a.nome like '*eira%'
```

O valor de um termo lógico pode ser *verdadeiro*, *falso* ou *desconhecido*. O valor lógico *desconhecido* decorre da existência, em bancos de dados, de valores nulos.³

OPERADORES LÓGICOS

Os termos lógicos são combinados por operadores lógicos, como **and**, **or** e o símbolo de negação **not**, com o auxílio de parênteses. Exemplos de expressões lógicas combinando dois ou mais termos lógicos são

```
codcurso = 'INF' and sexo = 'F'
( codcurso = 'INF' or codcurso = 'DIR' ) and sexo = 'F'
not codcurso = 'INF' and sexo = 'F'
```

O valor final de uma expressão lógica pode ser *verdadeiro*, *falso*, ou *desconhecido*. Esse valor é construído a partir do resultado da aplicação dos operadores lógicos sobre os valores dos termos lógicos que constituem a expressão. O quadro abaixo mostra o resultado da aplicação dos operadores lógicos mais comuns, de acordo com os diferentes valores de seus operandos. Os operadores *and* e *or* têm dois operandos e o operador de negação apenas um. *V* e *F* significam *verdadeiro* e *falso* respectivamente; *?* significa *desconhecido*.

V and V → V

V and F → F

V and ? → ?

F and F → F

F and ? → F

? and ? → ?

V or V → V

V or F → V

V or ? → V

F or F → F

F or ? → ?

? or ? → ?

³ A manipulação de valores nulos é tratada com detalhes na seção 4.9.

not V → F
not F → V
not ? → ?

Algumas implementações SQL suportam um conjunto de operadores lógicos estendido, formado pelos operadores básicos e operadores adicionais. Por exemplo, o Access disponibiliza o operador lógico **xor**, bastante usado em linguagens de programação. O efeito desse operador, contudo, pode ser emulado pela combinação dos operadores básicos, como mostrado a seguir.

$x \text{ xor } y$ é equivalente a $(x \text{ and } (\text{not } y)) \text{ or } ((\text{not } x) \text{ and } y)$

Assim, é importante compreender que qualquer desses operadores adicionais pode ser expresso através de uma expressão equivalente formada pelos operadores básicos acima. O objetivo dos operadores estendidos é, apenas, facilitar a escrita de certas expressões.

PRECEDÊNCIA DOS OPERADORES LÓGICOS

É importante observar a precedência dos operadores numa expressão lógica, pois ela indica a ordem em que os termos lógicos são avaliados, tal como numa expressão aritmética as operações de multiplicação e divisão precedem as operações de soma e subtração. Ainda semelhantemente a uma expressão aritmética, a ordem de precedência normal pode ser alterada pelo uso de parênteses (dos termos lógicos mais internos para os mais externos) na expressão SQL. Ao final deste capítulo, são apresentadas as regras de precedência para cada uma das implementações SQL abordadas. Note que nem sempre essas regras coincidem.

∴

O exemplo a seguir ilustra um engano bastante comum na construção de expressões lógicas, ocasionado pela não observância da precedência dos operadores. Vamos supor que seja preciso identificar os alunos nascidos após 01/01/1979, que sejam do curso de Direito ou do curso de Jornalismo.⁴



select nome, codcurso, nascimento
from aluno
where codcurso = 'DIR'
 or codcurso = 'JOR'
 and nascimento >= '01/01/1979'


select nome, codcurso, nascimento
from aluno
where codcurso = 'DIR'
 or codcurso = 'JOR'
 and nascimento >= '1979-01-01'


select nome, codcurso, nascimento
from aluno
where codcurso = 'DIR'
 or codcurso = 'JOR'
 and nascimento >= #01/01/1979#



nome	codcurso	nascimento
Ricardo Biondi	DIR	21/02/80
Oscarito Vianna	DIR	14/08/79
Barbara Carlito	JOR	29/10/79
Carlos Maradona	DIR	30/06/77

Esta forma da consulta SQL produz um resultado incorreto, pois inclui um aluno de Direito que nasceu em 1977. Isto ocorre porque o conector lógico **and** tem precedência sobre o conector ló-

⁴ No Access, literais de data devem estar delimitados pelo caracter #. No MySQL, as datas são escritas no formato yyyy-mm-dd. No Oracle e SQL Server, a ordem dos componentes pode ser customizada. Vide seções 4.15 até 4.18.

gico *or*, sendo avaliado antes. Para os alunos de Direito, não prevaleceu a condição de terem nascido após a data especificada. Na realidade, foram selecionados os alunos que

são do curso de Jornalismo e nasceram em ou depois de 01/01/1979
ou
são do curso de Direito.

Este erro é corrigido numa segunda versão, com a ajuda de parênteses.



```

select nome, codcurso, nascimento
from aluno
where ( codcurso = 'DIR'
      or
      codcurso = 'JOR' )
and nascimento >= '01/01/1979'

```



```

select nome, codcurso, nascimento
from aluno
where ( codcurso = 'DIR'
      or
      codcurso = 'JOR' )
and nascimento >= '1979-01-01'

```



```

select nome, codcurso, nascimento
from aluno
where ( codcurso = 'DIR'
      or
      codcurso = 'JOR' )
and nascimento >= #01/01/1979#

```



nome	codcurso	nascimento
Ricardo Biondi	DIR	21/02/80
Oscarito Vianna	DIR	14/08/79
Barbara Carlito	JOR	29/10/79
Carlos Maradona	DIR	30/06/77

Agora fica claro que devem ser selecionados alunos que

*são do curso de Jornalismo **ou** são do curso de Direito*
e
nasceram em ou depois de 01/01/1979.

∴

Qualquer termo ou expressão lógica pode ser precedido da negação. No exemplo a seguir, a condição de que o curso do aluno seja Direito ou Jornalismo aparece inteiramente negada, significando que o curso **não** deve ser Direito, **nem** Jornalismo, como mostra o resultado da consulta.

```

select nome,
       codcurso,
       nascimento
from aluno
where not ( codcurso = 'DIR' or codcurso = 'JOR' )

```



matricula	nome	nascimento
1002	Maria Rita Colatti	INF
1008	Sacaruda Miranda	INF

∴

Outro exemplo de negação lógica é a consulta abaixo, que produz os alunos que não nasceram antes da referida data.

C

S

select nome, nascimento
 from aluno
 where not nascimento < '01/01/1979'

M

select nome, nascimento
 from aluno
 where not nascimento < '1979-01-01'

A

select nome, nascimento
 from aluno
 where not nascimento < #01/01/1979#

nome

nascimento

Ricardo Biondi

21/02/80

Oscarito Vianna

14/08/79

Barbara Carlito

29/10/79

Sacadura Miranda

12/12/81

A negação também está sujeita às regras de precedência. Na consulta

 	select nome, nascimento from aluno where not nascimento < '01/01/1979' and codcurso = 'JOR'
	select nome, nascimento from aluno where not nascimento < '1979-01-01' and codcurso = 'JOR'
	select nome, nascimento from aluno where not nascimento < #01/01/1979# and codcurso = 'JOR'

não fica claro se o operador *and* seria aplicado antes ou depois do *not*. Se o operador *and* fosse o primeiro, a consulta seria equivalente a

 	select nome, nascimento from aluno where not (nascimento < '01/01/1979' and codcurso = 'JOR')
	select nome, nascimento from aluno where not (nascimento < '1979-01-01' and codcurso = 'JOR')
	select nome, nascimento from aluno where not (nascimento < #01/01/1979# and codcurso = 'JOR')

e o resultado seria

nome	codcurso	nascimento
Ricardo Biondi	DIR	21/02/80
Maria Rita Colatti	INF	10/11/78
Oscarito Vianna	DIR	14/08/79
Barbara Carlito	JOR	29/10/79
Carlos Maradona	DIR	30/06/77
Sacadura Miranda	INF	12/12/81

Por outro lado, se o operador *not* fosse avaliado antes do operador *and*, a consulta seria equivalente a

 	select nome, nascimento from aluno where (not nascimento < '01/01/1979') and codcurso = 'JOR'
	select nome, nascimento from aluno where (not nascimento < '1979-01-01') and codcurso = 'JOR'
	select nome, nascimento from aluno where (not nascimento < #01/01/1979#) and codcurso = 'JOR'

e o resultado seria

nome	codcurso	nascimento
Barbara Carlito	JOR	29/10/79

Em geral, o operador *not* tem precedência sobre o operador *and*, mas esses últimos exemplos mostram como uma mudança sutil na expressão lógica pode alterar significativamente o resultado de uma consulta. A lista abaixo apresenta a precedência dos operadores lógicos. Os operadores de maior precedência são avaliados antes. Entre dois operadores de mesma precedência, é avaliado primeiro o que estiver mais à esquerda na expressão.

Operador lógico	Precedência
not	3
and	2
or	1

Mais adiante neste capítulo, é apresentado o quadro completo de precedências, incluindo operadores lógicos e aritméticos, para cada implementação SQL.

4.3 OPERADOR BETWEEN

É muito comum o uso de expressões lógicas que testam se um valor pertence a um intervalo ou não. Vejamos a consulta que requer os nomes dos alunos que nasceram nos anos de 1978 e 1979.

  **select** nome
from aluno
where nascimento >= '01/01/1978' **and** nascimento <= '12/31/1979'

 **select** nome
from aluno
where nascimento >= '1978-01-01' **and** nascimento <= '1979-12-31'

 **select** nome
from aluno
where nascimento >= #01/01/1978# **and** nascimento <= #12/31/1979#

A data de nascimento deve estar no intervalo compreendido entre as datas referidas, incluindo os valores extremos. Com o auxílio do operador **between**, um termo lógico equivalente pode ser escrito como

  **select** nome
from aluno
where nascimento **between** '01/01/1978' **and** '12/31/1979'

 **select** nome
from aluno
where nascimento **between** '1978-01-01' **and** '1979-12-31'

 **select** nome
from aluno
where nascimento **between** #01/01/1978# **and** #12/31/1979#

O termo lógico pode ser construído de forma inversa logicamente. A consulta que retorna os alunos que **não** nasceram no referido intervalo de datas pode ser escrita como

  **select** nome
from aluno
where nascimento **not between** '01/01/1978' **and** '12/31/1979'

 **select** nome
from aluno
where nascimento **not between** '1978-01-01' **and** '1979-12-31'

 **select** nome
from aluno
where nascimento **not between** #01/01/1978# **and** #12/31/1979#

O mesmo efeito pode ser obtido pela negação do termo lógico por inteiro, como na consulta

  **select** nome
from aluno
where not nascimento **between** '01/01/1978' **and** '12/31/1979'

 **select** nome
from aluno
where not nascimento **between** '1978-01-01' **and** '1979-12-31'

 **select** nome
from aluno
where not nascimento **between** '01/01/1978' **and** #12/31/1979#

 O operador *between* pode ser utilizado com operandos de quaisquer tipos de dados, desde que possam ser comparados. A ordem dos extremos do intervalo pode afetar o resultado da consulta dependendo da implementação. A consulta abaixo define o intervalo iniciando pelo maior valor e produz resultados diversos, como mostrado.

 **select** *
from inscricao
where nota **between** 6 **and** 3

   **select** *
from inscricao
where nota **between** 6 **and** 3

→

matricula	coddisciplina	nota
1005	316	3
1005	117	4,2
1010	317	5,5

→

matricula	coddisciplina	nota
-----------	---------------	------

No Access, a consulta funciona normalmente, mostrando as inscrições com notas entre 3 e 6. Já para o MySQL, Oracle e SQL Server, a consulta produz um resultado vazio, isto é, nenhuma nota é considerada pertencente ao intervalo com os extremos invertidos. Isto ocorre porque o teste exige que o valor da expressão seja maior ou igual ao primeiro valor do intervalo e menor ou igual ao segundo.

4.4 OPERADOR IN

Outra construção comum nas expressões é a especificação de várias alternativas de valor, normalmente com o conector lógico *or*. A consulta

select nome
from aluno
where codcurso = 'DIR' **or** codcurso = 'JOR'

pode ser expressa como

```
select nome
from aluno
where codcurso in ('DIR', 'JOR')
```

com o auxílio do operador **in**, que caracteriza um termo lógico. Note que para duas ou três alternativas não há grande diferença, mas se for necessário escrever uma consulta com quatro, cinco, ou mais alternativas, o uso do operador *in* proporciona um texto bem mais legível.

A negação do operador *in* pode ser feita de duas formas. Negando o termo lógico por inteiro

```
select nome
from aluno
where not codcurso in ('DIR', 'JOR')
```

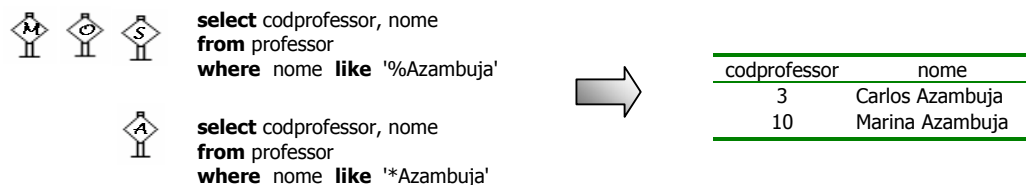
ou usando a construção que inverte logicamente o termo lógico, específica para o operador *in*

```
select nome
from aluno
where codcurso not in ('DIR', 'JOR')
```

Todas as alternativas devem ter tipos compatíveis com o valor sendo testado, como se fosse um conjunto de comparações feitas separadamente. O operador *in* pode ainda ser utilizado com subconsultas, como visto no capítulo 9. Além disso, a seção 4.8 detalha o comportamento deste operador na comparação com valores nulos.

4.5 OPERADOR LIKE

O operador **like**⁵ permite flexibilizar os critérios de filtragem na cláusula *where*. O operador *like*, que atribui significado aos caracteres curingas que formam padrões de comparação, trata-se na verdade de um operador de comparação especial, e portanto pode ser utilizado na construção de termos lógicos, como vistos acima. Para exemplificar sua utilização, vamos supor que seja necessário fazer uma busca para identificar os professores cujo sobrenome seja *Azambuja*.



Este caso ilustra o uso dos caracteres curingas em conjunto com o operador *like*. No padrão de comparação “%Azambuja”, o caracter % (ou o caracter * para o Access) casa com qualquer *string* de zero ou mais caracteres. Assim, para o nome *Carlos Azambuja*, presente no resultado, o caracter % casa com a string “Carlos “ e a expressão lógica torna-se verdadeira. Note que o caracter curinga foi utilizado do lado esquerdo do padrão, obrigando que a parte final do nome tivesse que ser, literalmente, a palavra *Azambuja*.

⁵ No MySQL, os comandos *rlike* e *regex* têm função semelhantes à do comando *like*.

Os caracteres curingas valem apenas para comparações com o operador **like**. Nas demais comparações strings, esses caracteres não têm função alguma, isto é, valem exatamente como estão escritos.

Há situações em que é necessário pesquisar a ocorrência de um ou mais desses caracteres em combinação com outros padrões. Suponha que se queira procurar por alguma string que contenha o caractere % em qualquer posição. Nesse caso, é preciso lançar mão do que é chamado **caracter de escape** (*escape character*, em inglês). O caractere de escape tem a função de desligar o efeito do caractere curinga que o sucede na string de comparação.

Por exemplo, se o caractere de escape for a barra invertida \ então a string de comparação “%\\%” casa com todas as strings que têm algum caractere % em qualquer posição. Isto porque o primeiro % significa qualquer string com zero ou mais caracteres; o segundo caractere %, porém, sucede o caractere de escape e não tem efeito curinga e representa o próprio caractere %; o terceiro % é um caractere curinga porque não é precedido pelo caractere de escape e também significa qualquer string.

Os caracteres curinga e a especificação de caracteres de escape variam com a implementação do SQL. As seções ao final deste capítulo mostram as particularidades dos sistemas gerenciadores de bancos de dados neste quesito.

4.6 RENOMEANDO AS COLUNAS DO RESULTADO

O resultado de uma consulta SQL é sempre representado na forma de uma tabela relacional, onde cada coluna tem seu próprio nome. O nome resultante de cada coluna normalmente advém do atributo que a preenche, como foi visto nos diversos exemplos até aqui. Entretanto, qualquer coluna do resultado de uma consulta pode ser renomeada.

```
select matricula as registro,  
       nome as estudante  
from aluno  
where CodCurso = 'DIR'
```



registro	estudante
1001	Ricardo Biondi
1004	Oscarito Vianna
1007	Carlos Maradona

Para se renomear uma coluna, basta utilizar a palavra-chave **as**⁶ seguida do nome desejado. No exemplo acima, as colunas *matricula* e *nome* foram renomeadas como *registro* e *estudante*, respectivamente.



RENOMEANDO COLUNAS NO SQL SERVER

O SQL Server permite que as colunas sejam renomeadas com uma sintaxe alternativa. A consulta acima poderia ser escrita como



```
select registro = matricula, estudante = nome  
from aluno  
where CodCurso = 'DIR'
```

Nesta sintaxe, o nome da coluna, acompanhado do símbolo “=”, precede a expressão que dá valor à coluna.

⁶ Obrigatória no Access e opcional nas demais implementações.

4.7 COLUNAS COM VALORES CALCULADOS

O resultado de uma consulta SQL pode conter uma ou mais colunas oriundas de operações aritméticas efetuadas sobre os atributos que compõem os registros. Vamos supor que seja preciso preparar a lista de disciplinas juntamente com o número de créditos de cada uma. Este dado não faz parte do banco, mas sabe-se que, no nosso exemplo, cada hora-aula teórica equivale a um crédito e cada hora-aula prática, a meio crédito. Assim, é possível calcular o número de créditos de cada disciplina.

```
select disciplina,
        chst,
        chsp,
        chst + (chsp * 0.5) as creditos
from disciplina
```



disciplina	chst	chsp	creditos
Dir. Constitucional	4	0	4
Direito Civil	4	2	5
Estatística	2	2	3
Compiladores	2	4	4
Bancos de Dados	4	2	5
Sociologia	3	1	3,5
Português	4	0	4

Normalmente, em todas as implementações SQL a ocorrência de valores nulos em algum dos fatores de uma expressão aritmética produz o valor nulo com resultado (vide seção 4.8).

∴



Note que a sintaxe alternativa para renomear colunas vale também para colunas com valores calculados, assim como em qualquer outra situação. A consulta acima pode ser escrita como



```
select disciplina,
        chst,
        chsp,
        Creditos = chst + (chsp * 0.5)
from disciplina
```

∴



Quando uma coluna do resultado não é formada diretamente a partir de algum atributo das tabelas da base de dados, a implementação SQL “inventa” um nome para essa coluna. Veja no final deste capítulo um exercício que explora as regras de formação de nomes para cada uma das implementações SQL aqui abordadas.

4.8 LIDANDO COM VALORES NULOS

Valores nulos recebem tratamento especial na avaliação de expressões lógicas. Vejamos o exemplo de uma consulta que obtém a lista de inscrições com notas iguais ou acima de 5,0.

```
select *
from inscricao
where nota >= 5.0
```



matricula	coddisciplina	nota
1001	317	8,0
1002	210	9,5
1007	114	7,0
1010	317	5,5
1010	316	10,0

No resultado, aparecem somente as disciplinas que têm graus superiores ou iguais a cinco. As disciplinas que não têm suas notas preenchidas não aparecem. É interessante agora comparar este resultado com o resultado do exemplo abaixo, onde são requeridas as inscrições que têm notas abaixo de 5,0.

```
select *
from inscricao
where nota < 5.0
```



matricula	coddisciplina	nota
1005	316	3,0
1005	117	4,2

Intuitivamente, podemos supor que todas as inscrições que não apareceram no exemplo anterior apareçam agora, ou seja, as inscrições que não têm notas iguais ou superiores a 5,0, pois a condição de filtro é exatamente oposta. Isso, entretanto, não acontece pois somente dois registros são mostrados no resultado acima. As duas inscrições cujas notas não foram preenchidas não aparecem em nenhuma das duas consultas, porque o mecanismo de avaliação de expressões lógicas do SQL designa o valor *desconhecido* para termos lógicos onde qualquer dos operandos tenha o valor nulo. Uma nota que não foi preenchida é desconhecida e, portanto, não se pode dizer que seja maior, igual ou menor que 5,0! Essa é a idéia que permeia o tratamento de nulos pelo SQL.

Essa característica constitui grande potencial de erros para o programador desavisado, pois consultas escritas corretamente podem omitir registros e levar a resultados confusos. No caso acima, se o programador contasse o número de inscrições com notas a partir de 5,0 numa consulta (5) e contasse o número de inscrições com nota abaixo de 5,0 em outra (2), o número total (5+2) não corresponderia ao total de inscrições (9), o que seria no mínimo estranho se não fosse fornecido o número de inscrições com notas ainda não preenchidas (2).

Mas como obter somente as inscrições que ainda não têm notas se os valores nulos têm restrições quanto ao operador de igualdade? Vejamos o exemplo abaixo.⁷

```
select *
from inscricao
where nota is null
```



matricula	coddisciplina	nota
1001	112	NULL
1007	112	NULL

A construção **is null** permite que o teste de nulidade seja feito explicitamente, de modo a identificar os valores nulos. Com este recurso, poderíamos incluir as inscrições com notas não preenchidas na lista dos que têm notas abaixo de 5,0.

⁷ Na apresentação das tabelas, valores nulos são explicitamente identificados como NULL.


```
select *
from inscricao
where ( nota < 5.0 ) or ( nota is null )
```



matricula	coddisciplina	nota
1	112	NULL
5	316	3,0
5	117	4,2
7	112	NULL

A palavra **null** é um literal que pode ser utilizado numa consulta SQL para incluir valores nulos nas linhas do resultado. Vejamos um exemplo.

```
select matricula,
       nome,
       null as passaporte
from aluno
```



matricula	nome	passaporte
1001	Ricardo Biondi	NULL
1002	Maria Rita Colatti	NULL
1004	Oscarito Vianna	NULL
1005	Barbara Carlito	NULL
1007	Carlos Maradona	NULL
1008	Sacadura Miranda	NULL
1010	Maria Lucia Silva	NULL

Neste caso, o resultado contém uma coluna, nomeada *passaporte*, que contém o valor nulo em todas as linhas. Este é um recurso bastante útil, especialmente em comandos SQL onde o resultado de uma consulta é utilizado para preencher tabelas da base de dados (vistos no capítulo 12).

O fato de se poder manipular o valor nulo explicitamente nas expressões SQL não altera em nada as regras de avaliação de expressões lógicas anteriormente estabelecidas. Assim, tanto a consulta

```
select *
from inscricao
where nota = null
```

como a consulta

```
select *
from inscricao
where nota <> null
```

produzem um resultado vazio, uma vez que nenhum registro da tabela INSCRIÇÃO torna verdadeira a expressão lógica. Isto ocorre porque, como visto anteriormente, um termo lógico que envolve algum operando cujo valor seja nulo resulta no valor lógico *desconhecido*.

∴

Outro aspecto importante na manipulação de valores nulos é seu efeito nas expressões aritméticas. O exemplo a seguir, que transforma as notas de uma escala de 0 a 10 para uma escala de 0 a 100, mostra que, para os registros onde o atributo *nota* é nulo, o resultado da multiplicação é também nulo.

```
select matricula,
       coddisciplina,
       nota * 10 as notasobre100
from inscricao
```



matricula	coddisciplina	notasobre100
1001	112	NULL
1001	317	80
1002	210	95
1005	316	30
1005	117	42
1007	112	NULL
1007	114	70
1010	317	55
1010	316	100

Uma vez que o resultado de uma expressão aritmética com nulos é também nulo, é possível testar o resultado da expressão com respeito à condição de nulidade.

```
select matricula,
       coddisciplina,
       nota * 10 as notasobre100
where nota * 10 is not null
from inscricao
```



matricula	coddisciplina	notasobre100
1001	317	80
1002	210	95
1005	316	30
1005	117	42
1007	114	70
1010	317	55
1010	316	100

∴

O que acontece numa consulta como

```
select nome
from aluno
where codcurso in ('DIR', 'JOR')
```

quando algum dos valores da lista pode ser nulo? Note que alguns dos valores na lista poderiam estar expressos pelo conteúdo de um atributo ou expressão. Por enquanto, vamos simular esta situação introduzindo explicitamente um valor nulo na lista, como abaixo.

```
select nome
from aluno
where codcurso in ('DIR', 'JOR', null)
```

O resultado esperado seria

nome
Ricardo Biondi
Oscarito Vianna
Barbara Carlito
Carlos Maradona
Maria Lucia Silva

ou seja, uma lista dos alunos que pertencem aos cursos de Direito e Jornalismo. Entretanto, a consulta com o operador **in** negado produziria, de acordo com a implementação, os resultados mostrados a seguir.



```
select nome
from aluno
where codcurso not in ('DIR', 'JOR', null)
```



nome
Maria Rita Colatti
Sacadura Miranda



```
select nome
from aluno
where codcurso not in ('DIR', 'JOR', null)
```



nome

Note que, para o Access, são mostrados os dois alunos que pertencem ao curso de Informática. Já para MySQL, Oracle e SQL Server, o resultado é vazio. Isso ocorre porque o valor nulo é considerado como uma incerteza. Ou seja, o resultado da comparação entre “INF” e o valor nulo é *desconhecido*, e não *falso*. Isto é facilmente compreensível se pensarmos que a última consulta acima é equivalente a

```
select nome
from aluno
where not (codcurso = 'DIR' or codcurso = 'JOR' or codcurso = null )
```

Para um aluno da Informática, cujo codcurso é “INF”, os três termos lógicos teriam os valores *falso*, *falso* e *desconhecido* respectivamente. Aplicando-se as regras da seção 4.2 para os operadores lógicos *or* e *not*, tem-se que o resultado final da expressão lógica é *desconhecido*, como demonstra a avaliação da expressão a seguir.

not (? or ? or ?) not (? or ?) not (?) not ? ?

∴



As regras que regem a avaliação de um termo lógico baseado no operador in cuja lista de valores contém nulos é a seguinte:

- se o valor procurado na lista é encontrado, o termo lógico tem valor *verdadeiro*;
- se o valor procurado não consta da lista e esta contém algum valor nulo, o termo lógico tem valor *desconhecido*.

∴



No MySQL, o operador `<=>` é equivalente ao operador `=`, porém sendo seguro em relação à ocorrência de valores nulos. Normalmente, quando um dos operadores numa igualdade tem valor nulo o termo lógico recebe o valor *desconhecido*. Isso ocorre mesmo que os dois operadores tenham valor nulo. Com a utilização do operador “`<=>`”, o resultado de um termo lógico pode ser *verdadeiro* ou *falso* apenas, sendo que a comparação de valores nulos é levada a efeito como se fossem valores comuns. Assim, a consulta

```
select *
from inscricao
where nota is null
```

é equivalente a



```
select *
from inscricao
where nota <=> null
```



No SQL Server, os operadores de comparação “`=`” e “`<>`” também podem ser seguros em relação à ocorrência de valores nulos caso a opção `ANSI_NULLS` seja desligada. O comando T-SQL

set ansi_nulls off

tem esse efeito. Nessas condições, os termos lógicos que contêm tais operadores retornam somente *verdadeiro* ou *falso*, ocorrendo ou não valores nulos na comparação.

4.9 CONCATENANDO ATRIBUTOS

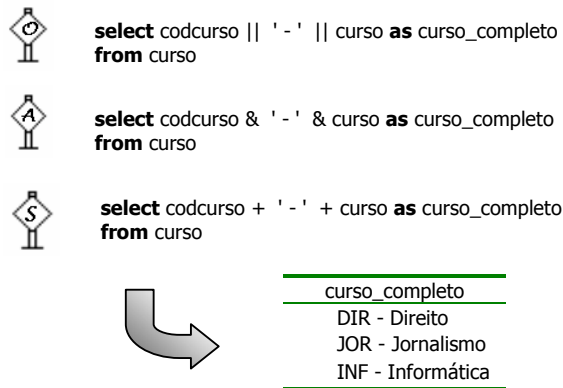
Diferentes atributos podem ser mesclados numa única coluna no resultado final, configurando um recurso de formatação bastante útil. O exemplo a seguir mostra um caso onde se requer a lista dos códigos de cursos e seus respectivos nomes.

```
select codcurso,
       '-' as hifen,
       curso
from curso
```



codcurso	hifen	curso
DIR	-	Direito
JOR	-	Jornalismo
INF	-	Informática

A seguir, o mesmo exemplo com os atributos concatenados.



O resultado possui uma única coluna, que é a concatenação do atributo *codcurso* com a *string* – (hífen) e o atributo *nome*.



CONCATENANDO ATRIBUTOS NO ACCESS

O símbolo normalmente utilizado para concatenação de strings é o caracter `&` embora o caracter `+` também possa ser utilizado para esse fim. Os atributos sendo concatenados são automaticamente convertidos para o tipo string. Se todos os atributos têm valor nulo, o resultado da concatenação é nulo. Se algum dos atributos é nulo, o mesmo é tratado como se fosse uma string vazia (com zero caracteres). Deve-se notar que, sendo a conversão de tipos automática, a utilização do caracter `+` como concatenador não é aconselhável, notadamente com números; `5 & 6` resulta na string “56”, mas `5 + 6` resulta no número 11! Dependendo do contexto e dos valores, é possível que o SQL comporte-se de uma forma que não exatamente a imaginada pelo programador.



CONCATENANDO ATRIBUTOS NO MYSQL

Normalmente, o MySQL não dispõe de um operador explícito para a concatenação de atributos. Esse efeito, no entanto, pode ser obtido pela utilização das funções **concat** e **concat_ws**, cuja aplicação é explicada com detalhes no Capítulo 8. Entretanto, quando o MySQL é executado com a opção **–ansi**, os caracteres `||` (duas barras verticais) passam a ser considerados como o símbolo de concatenação de strings. O mesmo efeito é obtido pela opção **--sql-mode = PIPES_AS_CONCAT**.

No MySQL, o exemplo anterior poderia ter sido escrito como



```
select concat (concat (codcurso, ' - ' ), curso) as curso_completo
from curso
```

ou utilizando barras verticais, como mencionado acima.



CONCATENANDO ATRIBUTOS NO ORACLE

O operador de concatenação é formado por duas barras verticais `||`. Operandos numéricos são automaticamente convertidos para strings.

CONCATENANDO ATRIBUTOS NO SQL SERVER

 O caracter + representa o operador de concatenação no SQL Server. Entretanto, o programador deve estar atento para a conversão de tipos, pois o SQL Server não converte automaticamente os operandos de uma concatenação e provoca um erro quando algum dos operadores não é de um tipo compatível com a operação.

Os operandos de uma concatenação devem possuir um tipo da classe de caracteres ou binário. Operandos de outros tipos devem ser explicitamente convertidos para um desses tipos, via alguma das funções de conversão apresentadas no capítulo 8. Isso ocorre porque o caracter + pode ser interpretado também como o operador aritmético de adição.

4.10 ORDENANDO O RESULTADO

O resultado de uma consulta SQL pode ter suas linhas ordenadas pelos valores de uma ou mais colunas, visíveis ou não neste resultado, através da cláusula **order by**. Cada coluna pode ser ordenada em ordem crescente ou decrescente, de acordo com as diretivas **asc** e **desc**, respectivamente.

```
select *
from aluno
order by nome asc
```



matricula	nome	sexo	codcurso	nascimento
1005	Barbara Carlito	F	JOR	29/10/79
1007	Carlos Maradona	M	DIR	30/06/77
1010	Maria Lucia Silva	F	JOR	10/08/75
1002	Maria Rita Colatti	F	INF	10/11/78
1004	Oscarito Vianna	M	DIR	14/08/79
1001	Ricardo Biondi	M	DIR	21/02/80
1008	Sacadura Miranda	M	INF	12/12/81

No exemplo acima, o resultado é construído em ordem crescente pela coluna *nome*. Um exemplo semelhante, com duas colunas no critério de ordenação, é mostrado a seguir.

```
select *
from aluno
order by codcurso, nome
```



matricula	nome	sexo	codcurso	nascimento
1007	Carlos Maradona	M	DIR	30/06/77
1004	Oscarito Vianna	M	DIR	14/08/79
1001	Ricardo Biondi	M	DIR	21/02/80
1002	Maria Rita Colatti	F	INF	10/11/78
1008	Sacadura Miranda	M	INF	12/12/81
1005	Barbara Carlito	F	JOR	29/10/79
1010	Maria Lucia Silva	F	JOR	10/08/75

Agora, o resultado aparece ordenado por *codcurso* e *nome*. Quando há mais de uma coluna no critério de ordenação, as linhas são ordenadas pela primeira delas, de acordo com a diretiva escolhida. Cada uma das demais colunas é ordenada, de acordo com sua própria diretiva, como se fosse o critério de desempate da coluna que a precede. Se nenhuma diretiva for utilizada para uma coluna, assume-se que a ordenação seja crescente.

Um critério de ordenação pode ser de três tipos:

- uma expressão;
- o nome de uma coluna;
- a posição relativa de uma coluna da lista-alvo.

O quadro abaixo ilustra esses três tipos, mostrando consultas com diferentes nuances sintáticas, mas com resultados idênticos: as disciplinas listadas em ordem decrescente de carga horária total, isto é, teórica mais prática, e pelo nome, em ordem crescente, como critério de desempate.

	<pre>select disciplina, chst + chsp as carga_horaria from disciplina order by chsp + chst desc, disciplina</pre>		<table border="1"> <thead> <tr> <th>disciplina</th><th>carga_horaria</th></tr> </thead> <tbody> <tr><td>Bancos de Dados</td><td>6</td></tr> <tr><td>Compiladores</td><td>6</td></tr> <tr><td>Direito Civil</td><td>6</td></tr> <tr><td>Dir. Constitucional</td><td>4</td></tr> <tr><td>Estatística</td><td>4</td></tr> <tr><td>Português</td><td>4</td></tr> <tr><td>Sociologia</td><td>4</td></tr> </tbody> </table>	disciplina	carga_horaria	Bancos de Dados	6	Compiladores	6	Direito Civil	6	Dir. Constitucional	4	Estatística	4	Português	4	Sociologia	4
disciplina	carga_horaria																		
Bancos de Dados	6																		
Compiladores	6																		
Direito Civil	6																		
Dir. Constitucional	4																		
Estatística	4																		
Português	4																		
Sociologia	4																		
<pre>select disciplina, chst + chsp as carga_horaria from disciplina order by carga_horaria desc, disciplina</pre>																			
<pre>select disciplina, chst + chsp as carg_horaria from disciplina order by 2 desc, 1</pre>																			

 A primeira consulta usa uma expressão como um dos critérios de ordenação. A segunda consulta utiliza o nome de uma coluna na cláusula *order by*. Como existe, na lista-alvo, uma coluna com a mesma expressão utilizada no critério de ordenação, não é preciso repeti-la, bastando referenciá-la pelo próprio nome. Esta sintaxe não é aceita no Access. Na terceira consulta, como as duas colunas aparecem na lista-alvo, basta colocar a posição relativa de cada uma na lista. A coluna que contém o total da carga horária é a segunda e a coluna com o nome da disciplina é a primeira.

Observe que os dois primeiros exemplos desta seção, onde são utilizados atributos simples no critério de ordenação (*nome* e *codcurso*), são casos particulares do primeiro e do segundo tipo, uma vez que podemos interpretar aquelas colunas tanto como: 1) expressões de um único termo; 2) colunas identificadas pelo nome.

Um detalhe importante é que os critérios de ordenação não devem, necessariamente, fazer parte da lista-alvo. Assim, uma consulta como a que é mostrada no próximo exemplo é válida, mesmo que a coluna do critério de ordenação não seja visível.

<pre>select disciplina from disciplina order by chst + chsp, coddisciplina</pre>		<table><tr><th>disciplina</th></tr><tr><td>Dir. Constitucional</td></tr><tr><td>Estatística</td></tr><tr><td>Sociologia</td></tr><tr><td>Português</td></tr><tr><td>Direito Civil</td></tr><tr><td>Compiladores</td></tr><tr><td>Bancos de Dados</td></tr></table>	disciplina	Dir. Constitucional	Estatística	Sociologia	Português	Direito Civil	Compiladores	Bancos de Dados
disciplina										
Dir. Constitucional										
Estatística										
Sociologia										
Português										
Direito Civil										
Compiladores										
Bancos de Dados										

Numa mesma consulta é permitido combinar os três tipos de referências para diferentes critérios de ordenação, como no exemplo abaixo.

```
select disciplina,
       chst + chsp as carga_horaria
from disciplina
order by chst + chsp desc, 1
```

Nos capítulos 6 e 8 são apresentadas outras possibilidades para a construção de expressões que sejam válidas como critérios de ordenação na cláusula *order by*.



ORDENANDO O RESULTADO NO ORACLE

A cláusula *order by* no Oracle admite algumas variações, como mostra a sintaxe abaixo.

```
<cláusula order by> :=
order [ siblings ] by
{
  { <expressão> | <posição da coluna> | <nome da coluna> }
  [ asc | desc ]
}
```

[**nulls first** | **nulls last**]

},...

Há uma opção que permite especificar se os valores nulos aparecem antes ou depois dos demais valores na ordem de classificação. Por exemplo, a consulta

```
select *
from inscricao
order by nota desc nulls last
```

produz como resultado

matricula	coddisciplina	nota
1010	316	10
1002	210	9,5
1001	317	8
1007	114	7
1010	317	5,5
1005	117	4,2
1005	316	3
1001	112	NULL
1007	112	NULL

Se nada for especificado, os valores nulos aparecem por último na ordem ascendente e no início na ordem descendente.



Note que, numa mesma consulta, é possível ter colunas ordenadas com nulos na frente e colunas com nulos no final. As opções *null first* e *null last* aplicam-se a uma coluna de cada vez.

∴

A opção *siblings* aplica-se a consultas hierárquicas e é objeto do exercício 11.3.

4.11 RESULTADOS SEM DUPLICATAS

Embora o resultado de uma consulta SQL seja, teoricamente, um conjunto, e portanto não deva admitir repetições, na prática as repetições não são removidas automaticamente, a menos que isso seja explicitamente determinado. Isso ocorre porque, em muitas situações, pode ser necessário que as duplicações sejam expressamente mantidas.

A presença da palavra-chave **distinct**, logo após a palavra *select*, determina que as duplicações devem ser retiradas. Vamos supor que seja necessário produzir a lista de códigos de disciplinas para as quais existe algum aluno inscrito. Nesse caso, as duplicações não são relevantes e seria melhor removê-las do resultado, como exemplificado abaixo.

```
select distinct coddisciplina
from inscricao
```



coddisciplina
112
114
117
210
316
317

Normalmente, o resultado de consultas que removem duplicatas sai ordenado na ordem natural das colunas, uma vez que o algoritmo que executa a extração de duplicações ordena todas as linhas para identificar as que são duplicatas.



No Oracle, a palavra-chave **unique** pode ser utilizada como sinônimo de *distinct*.



No MySQL, a palavra-chave **distinctrow** é sinônimo de *distinct*.



A OPÇÃO DISTINCTROW NO ACCESS

O Access oferece uma alternativa adicional no controle de duplicatas no resultado. Com a opção **distinctrow** é possível selecionar, no resultado, dados que tenham como origem registros diferentes ao invés de dados com valores diferentes.

Para ilustrar essa afirmativa, vamos considerar que o conteúdo da tabela ALUNO da base de exemplo tenha sido alterado, e apresenta-se como abaixo. Três registros foram acrescentados ao conteúdo original (matrículas 1014, 1015, 1017) e agora podemos observar a presença de dois pares de alunos homônimos, Oscarito Vianna e Maria Aparecida da Silva.

Matricula	Nome	Sexo	CodCurso	Nascimento
1001	Ricardo Biondi	M	DIR	21/02/80
1002	Maria Rita Colatti	F	INF	10/11/78
1004	Oscarito Vianna	M	DIR	14/08/79
1005	Barbara Carlito	F	JOR	29/10/79
1007	Carlos Maradona	M	DIR	30/06/77
1008	Sacadura Miranda	M	INF	12/12/81
1010	Maria Lucia Silva	F	JOR	10/08/75
1014	Maria Aparecida da Silva	F	JOR	24/09/77
1015	Oscarito Vianna	M	INF	12/05/79
1017	Maria Aparecida da Silva	F	INF	07/04/78

A consulta

```
select distinct nome
from aluno
order by nome
```



nome
Barbara Carlito
Carlos Maradona
Maria Aparecida da Silva
Maria Lucia Silva
Maria Rita Colatti
Oscarito Vianna
Ricardo Biondi
Sacadura Miranda

produz um resultado sem duplicações porque as linhas iguais são eliminadas. Note que os dois homônimos apareceram uma vez cada um. Já para a consulta

```
select distinctrow nome
from aluno
order by nome
```

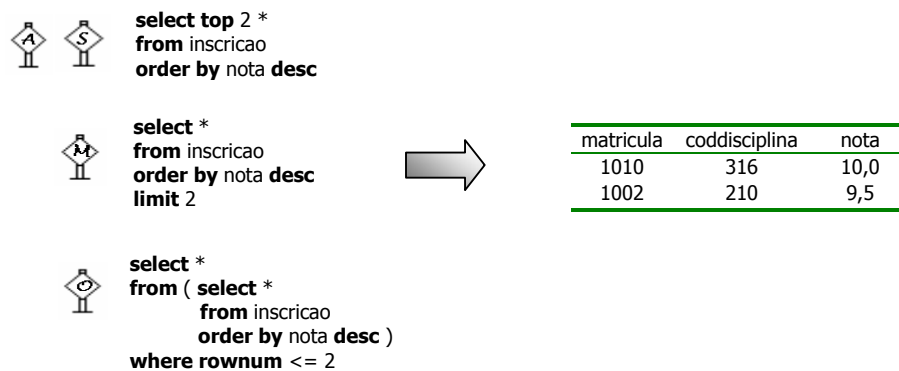


nome
Barbara Carlito
Carlos Maradona
Maria Aparecida da Silva
Maria Aparecida da Silva
Maria Lucia Silva
Maria Rita Colatti
Oscarito Vianna
Oscarito Vianna
Ricardo Biondi
Sacadura Miranda

temos um resultado com cada homônimo aparecendo duas vezes. A diferença é que, na segunda opção, são consideradas todas as linhas do resultado originadas de registros diferentes da base de dados, mesmo que sejam idênticas a alguma outra. Como cada homônimo advém de um registro separado, porque os homônimos têm diferentes matrículas, o resultado contém o mesmo nome mais de uma vez. Note que, no caso de *distinct*, a eliminação de linhas duplicadas é feita com base tão somente nos valores do resultado.

4.12 LIMITANDO O NÚMERO DE LINHAS DO RESULTADO

Em muitas situações, é interessante limitar o número de linhas num resultado. Isto ocorre, por exemplo, quando uma consulta está sendo construída e testada, fazendo com que apenas algumas poucas linhas do resultado sejam produzidas, de modo a simplificar e tornar mais rápido o processo de *debug*. Em outras situações, pode-se usar a limitação como um recurso do SQL. Vejamos um exemplo no qual se requer somente os dados das duas inscrições com as maiores notas.



A limitação de linhas, em geral, é aplicada somente no conteúdo final do resultado e depende da sua ordenação. Como a cláusula *where* não foi especificada no caso acima, todos os registros de INSCRIÇÃO são ordenados em ordem decrescente de nota mas somente os dois primeiros aparecem no resultado.

O critério de ordenação é importante quando as linhas são limitadas. O exemplo anterior, sem a cláusula *order by*, pode ter resultados imprevistos, pois a ordem dos registros numa consulta sem critério de ordenação depende de inúmeros fatores, e em muitos casos não pode ser prevista.

∴

 Para o Oracle, a construção para limitação de linhas é menos intuitiva que as demais, e envolve o emprego da função **rownum**. Veja maiores detalhes em seção específica mais adiante neste capítulo.

∴

 A limitação das linhas nem sempre traz uma redução significativa no tempo de execução de uma consulta. No exemplo acima, se a tabela INSCRICAO tivesse um milhão de registros poderíamos ter um longo tempo de execução, uma vez que todos os registros teriam que ser ordenados, mesmo para mostrar apenas duas linhas. Em contrapartida, em bancos de dados com tabelas muito grandes, esse recurso é especialmente útil quando se está construindo e testando consultas complexas. Contudo, a limitação sem ordenação poupa tempo na conferência dos resultados nos casos em que alguns poucos registros, mesmo que aleatoriamente escolhidos, são suficientes para a detecção de erros.



LIMITANDO O NÚMERO DE LINHAS DO RESULTADO NO ACCESS

O Access permite a limitação de linhas no resultado através da cláusula **top**, que especifica o número máximo (ou o percentual máximo em relação ao total) de linhas que a consulta deve produzir. Assim, a consulta

```
select top 2 *
from inscricao
order by nota desc
```

poderia ser expressa como

```
select top 20 percent *
from inscricao
order by nota desc
```

resultando numa limitação equivalente. Se, pelo critério de ordenação, houver empate em duas ou mais linhas do resultado, o Access pode exceder o limite incluindo tantas linhas quantas estiverem nessa condição.



LIMITANDO O NÚMERO DE LINHAS DO RESULTADO NO MYSQL

A cláusula **limit** é empregada no MySQL para limitar as linhas num resultado. Uma interessante característica desta cláusula é a possibilidade de indicar um *offset* inicial. Por exemplo, a consulta

```
select *
from inscricao
order by nota desc
limit 4,2
```

especifica que serão mostradas duas linhas a partir da quarta. Se apenas um número *n* é fornecido, MySQL assume que o *offset* é 1 e que *n* linhas serão listadas a partir da primeira.

Outra possibilidade interessante é utilizar **limit 0**. Neste caso, a consulta retorna vazia, imediatamente, o que é útil para rapidamente analisar a forma do resultado e o tipo de cada atributo ali presente.

A variável global **sql_select_limit** também pode ser utilizada para limitar linhas. Caso seja definida, a cláusula **limit** tem precedência sobre o valor dessa variável.



LIMITANDO O NÚMERO DE LINHAS DO RESULTADO NO ORACLE

O Oracle não dispõe de uma cláusula específica para limitar o número de linhas. Efeito semelhante, entretanto, pode ser obtido pela inclusão de um termo lógico específico na cláusula **where** que determine a limitação. Toda consulta tem uma pseudocoluna chamada **rownum** que contém o número de ordem na qual cada linha do resultado foi obtida. Note que *rownum* é aplicada de acordo com a ordem dos registros na fonte de registros.

No exemplo da seção 4.13, a consulta

```
select *
from ( select *
      from inscricao
      order by nota desc )
where rownum <= 2
```

mostra a utilização deste recurso.



Qualquer termo lógico que teste se **rownum** é maior do que um número inteiro é falso, uma vez que isso tornaria inviável a construção do resultado em alguns casos. Veja maiores detalhes sobre a pseudocoluna *rownum* no capítulo 11.

∴

Outro caminho para a obtenção de resultados abreviados no Oracle é a cláusula *sample*, que pode ser embutida na cláusula *from*. Com isso, pode-se determinar que apenas uma certa percentagem dos registros de uma tabela, utilizada como fonte de registros, participe de uma consulta. Na consulta

```
select *
from inscricao sample (50)
```

o resultado produzido considera apenas 50% dos registros da tabela INSCRIÇÃO, escolhidos aleatoriamente. Esta construção, entretanto, funciona apenas para consultas com apenas uma fonte de registros. Esta opção é bastante útil na visualização de uma tabela muito extensa, por amostragem.



LIMITANDO O NÚMERO DE LINHAS DO RESULTADO NO SQL SERVER

A exemplo do que foi visto para o Access, o SQL Server permite a limitação de linhas pela cláusula **top**. Contudo, quando há empate no critério de ordenação entre duas ou mais linhas, é possível optar pela inclusão de todas as linhas empatadas ou pelo corte no número exato. A consulta

```
select top 8 with ties *
from inscricao
order by nota desc
```

produz um resultado com 9 linhas, porque há duas notas nulas empatadas, que seriam a oitava e nona linhas. A opção **with ties** indica que as linhas empatadas devem ser todas listadas. Já a consulta

```
select top 8 *
from inscricao
order by nota desc
```

produz um resultado com exatas oito linhas, não sendo previsível qual das duas que estão empatadas aparece e qual deixa de aparecer.

Outra forma de limitar o número de linhas, embora indireta, é executar o comando T-SQL

```
set rowcount n
```

antes da consulta, onde *n* é o número máximo desejado.

4.13 COLUNAS COM VALORES CONDICIONAIS

Nos exemplos vistos até aqui, os valores de cada coluna do resultado têm valores por expressões. Ou são originados de algum atributo diretamente, ou são combinados em expressões aritméticas ou lógicas, mas sempre de forma uniforme para todas as colunas. Entretanto, o SQL permite que uma ou mais colunas do resultado possuam valores originados de diferentes expressões, tal que a escolha seja determinada a partir de expressões lógicas.

Para tanto, MySQL, Oracle e SQL Server permitem o emprego do construtor **case ... end**, ilustrado a seguir por dois exemplos. Quanto ao Access, há uma função similar, apresentado na próxima seção.

Um primeiro exemplo apresenta a lista de alunos com o atributo sexo escrito por extenso.



```

select nome,
       case sexo when 'M' then 'Masculino'
              when 'F' then 'Feminino'
              else 'Não preenchido'
       end as sexo_por_extenso
from aluno

```



nome	sexo_por_extenso
Ricardo Biondi	Masculino
Maria Rita Colatti	Feminino
Oscarito Vianna	Masculino
Barbara Carlito	Feminino
Carlos Maradona	Masculino
Sacadura Miranda	Masculino
Maria Lucia Silva	Feminino

Dependendo do valor do atributo *sexo*, a segunda coluna pode ser preenchida com diferentes strings. Após a palavra *case* aparece a expressão cujo valor deve ser testado. No nosso exemplo, essa expressão é composta por um simples atributo. Os testes, identificados pela palavra-chave *when*, são feitos em seqüência, isto é, falhando um passa-se para o próximo até que algum seja satisfeito. A cláusula *else* é opcional, e pode ser utilizado quando nem todos os possíveis valores da expressão precisam ser testados.

Note que, neste caso, há um teste de nulidade: se o atributo não estiver preenchido, o valor final da coluna é “Não preenchido”. Para nossa base de teste, que não possui registros com este atributo não preenchido, o mesmo resultado final seria obtido com



```

select nome,
       case sexo when 'M' then 'Masculino'
              when 'F' then 'Feminino'
       end as sexo_por_extenso
from aluno

```

ou com



```

select nome,
       case sexo when 'M' then 'Masculino'
              else 'Feminino'
       end as sexo_por_extenso
from aluno

```

Na primeira opção, omitimos a cláusula *else*, que é opcional nesta construção. Na segunda opção, o *else* foi usado diretamente em caso de falha do teste com o valor “M”.

Um segundo exemplo trata de um resultado no qual as notas dos alunos aparecem expressas na forma de conceitos. Notas iguais ou maiores que 9,0 equivalem ao conceito A; notas menores que 9,0 e iguais ou maiores que 7,0 equivalem ao conceito B; notas menores que 7,0 e iguais ou maiores que 5,0 equivalem ao conceito C; notas menores que 5,0 equivalem ao conceito D. Notas com valores nulos devem ser apropriadamente assinaladas.

```

select matricula,
        coddisciplina,
        nota,
        case when nota >= 9.0 then 'A'
          when nota >= 7.0 then 'B'
          when nota >= 5.0 then 'C'
          when nota >= 0.0 then 'D'
          else 'Não preenchida'
        end as conceito
from inscricao

```



matricula	coddisciplina	nota	conceito
1001	112	NULL	Não preenchida
1001	317	8.0	B
1002	210	9.5	A
1005	316	3.0	D
1005	117	4.2	D
1007	112	NULL	Não preenchida
1007	114	7.0	B
1010	317	5.5	C
1010	316	10.0	A

A segunda forma sintática do **case...end** é mais adequada quando a lista de valores possíveis não é conhecida, ou é formada por intervalos, como no caso do exemplo das notas, que são apresentadas como conceitos. Aqui, não há um valor específico sendo testado, mas uma série de expressões lógicas. A primeira que for verdadeira quebra a sequência e a encerra. Se nenhuma for verdadeira, prevalece o *else*. Se nenhuma das expressões for verdadeira e não houver *else*, que é opcional, o resultado do *case* é nulo.

∴

Resumidamente, a principal diferença entre as duas formas sintáticas do construtor *case ... end* é que na primeira forma a expressão testada é sempre a mesma, e normalmente são identificados valores discretos, como numa lista; na segunda forma, não há uma expressão específica sendo, podendo-se utilizar diversas expressões lógicas, sendo assim mais apropriadas para teste de intervalos ou quando há mais de uma expressão a ser considerada.

COLUNAS COM VALORES CONDICIONAIS NO ACCESS

No Access, colunas com valores condicionais são tratadas pela função **iif** (*if imediato* ou *immediate if*). Para o primeiro exemplo, a consulta deve ser escrita como

```

select nome,
        iif (sexo = 'M', 'Masculino', 'Feminino') as sexo_por_extenso
from aluno

```

e, para o segundo, como

```

select matricula,
        coddisciplina,
        iif (nota>=9.0,'A', iif (nota >= 7.0,'B', iif (nota >= 5.0,'C', iif (nota >= 0.0,'D','Não preenchida'))))
        as conceito
from inscricao

```

A estrutura da função *iif* é bastante simples. A primeira parte é uma expressão lógica e a segunda e a terceira partes são expressões simples. Se a expressão lógica for verdadeira, a função re-

torna o valor da segunda expressão; se for falsa ou desconhecida, retorna o valor da terceira expressão. Assim, a função

```
iif (sexo = 'M', 'Masculino', 'Feminino')
```

retorna “Masculino” (valor da segunda expressão) quando sexo = “M” e “Feminino” (valor da terceira expressão), caso contrário. Note que se o atributo *sexo* fosse nulo, a função retornaria “Feminino”. Na consulta do segundo exemplo, a mesma função é utilizada, porém com várias chamadas “aninhadas”.

```
iif (nota >= 9.0, 'A', iif (nota >= 7.0, 'B', iif (nota >= 5.0, 'C', iif (nota < 5.0, 'D', 'Não preenchida'))))
```

As linhas acima ilustram o aninhamento das funções. Cada linha sublinha a terceira parte de um *if imediato*. No primeiro *if*, se a expressão lógica *nota >= 9.0* é falsa, então o resultado passa a ser o resultado de um outro *if imediato*, embutido no primeiro, e assim sucessivamente até o último. Note que a última expressão serve para testar a ocorrência de nulos. Se uma nota chega até o quarto *if*, significa que ela tem que ser menor que 5,0, senão teria caído em alguma condição anterior. O teste final *nota < 5.0* serve para garantir que a nota não é nula. Em sendo nula, este termo de comparação tem o valor lógico *desconhecido*.



COLONAS COM VALORES CONDICIONAIS NO MYSQL

No MySQL, além do construtor *case..end*, pode-se utilizar a função **if**, que funciona exatamente como a função *iif* mostrada para o Access, exceto pela grafia. Para o primeiro exemplo, deve ser escrita como



```
select nome,  
       if (sexo = 'M', 'Masculino', 'Feminino') as sexo_por_extenso  
from aluno
```

e, para o segundo exemplo, como



```
select matricula,  
       coddisciplina,  
       if (nota >= 9.0, 'A', if (nota >= 7.0, 'B', if (nota >= 5.0, 'C', if (nota >= 0.0, 'D', 'Não preenchida'))))  
       as conceito  
from inscrição
```



COLONAS COM VALORES CONDICIONAIS NO ORACLE

O construtor *case..end* foi introduzido no Oracle nas versões mais recentes. Até então, a função **decode** era a opção natural para programadores SQL.

O primeiro exemplo, que trata da lista de alunos e respectivos sexos por extenso, pode ser resolvido assim.



```
select nome,  
       decode ( sexo,  
               'M', 'Masculino',  
               'F', 'Feminino',  
               'Não preenchido') as sexo_por_extenso  
from aluno
```

A expressão *decode* trabalha com o valor que deve ser decodificado (no caso, o atributo *sexo*), uma lista de pares de valores e o valor *default*. Cada par contém um valor de busca e um valor resultante que lhe corresponde. No presente caso, esses valores indicam que se o valor do atributo *sexo* for “M”, então o valor resultante da expressão *decode* é “Masculino”; se for “F”,

“Feminino”. O último valor, *default*, é o valor que a expressão assume se nenhum dos valores de busca tiver sido encontrado. No caso da expressão ser escrita sem o valor *default*, a expressão retorna nulo se não houver coincidência com os valores de busca, como abaixo.



```
select nome,
       decode ( sexo, 'M', 'Masculino', 'F', 'Feminino' ) as sexo_por_extenso
from aluno
```

4.14 LITERAIS E OPERADORES NO ACCESS



LITERAIS NO ACCESS

Números

Números são escritos normalmente no Access. Nos scripts SQL, Access não é sensível às configurações regionais. Por exemplo, uma função de conversão de tipo como *CDBl*, que converte de *string* para número real (vista no capítulo 8), observa as configurações vigentes e deve ser escrita de acordo com elas. O exemplo abaixo ilustra este caso.

```
select CDBl ('5,2') as x, 5.2 + 1 as y
```

x	y
5,2	6,2

Note que, neste caso, supõe-se que o caractere decimal escolhido seja a vírgula.

Datas e horários

Além do formato numérico, Access aceita datas numa codificação onde os meses são designados nominalmente, como no comando a seguir.

```
select CDate ('21-ago-2010') as x, #21-aug-2010# as y
```

x	y
21/08/2010	21/08/2010

Note que a sigla para cada mês depende do idioma que prevalece na instalação. No caso acima, ago é reconhecido como o mês de agosto porque a configuração supõe o idioma português. Nas datas entre os caracteres # e #, contudo, vale sempre a configuração em inglês, o que exige a sigla *aug*, como em *august*. Se invertêssemos os valores haveria erros nos dois casos.

∴

No Access, as configurações regionais do sistema operacional podem gerar situações de conflito se não forem cuidadosamente consideradas. Observe o comando SQL a seguir.

```
select CDate ('12/11/1980') as x, #12/11/1980# as y
```

No Access, o resultado produzido por esta consulta depende da configuração regional vigente. Como a função *CDate* (converte de um tipo qualquer para data) é sensível à configuração regional, o resultado pode ser

x	y
12/11/1980	11/12/1980

se o formato de data for dd/mm/yyyy, ou

x	y
11/12/1900	11/12/1900

se o formato de data for mm/dd/yyyy (note que a segunda data não muda). Isso ocorre porque as datas entre os caracteres # são preferencialmente interpretadas no formato mm/dd/yyyy; para a função *CDate*, valem preferencialmente as configurações regionais vigentes. No resultado, as datas são sempre mostradas de acordo com a configuração vigente.

∴



No exemplo acima, o termo preferencialmente foi propositalmente utilizado. O mesmo comando, agora com datas inválidas, executado na mesma implementação, produziria re-

select CDate('12/31/1980') **as** x, #12/31/1980# **as** y

x	y
31/12/1980	31/12/1980

sultados distintos, haja vista que as mesmas foram interpretadas numa ordem exatamente inversa daquela acima.

Hexadecimais

Literais para caracteres hexadecimais podem ser utilizadas no Access quando precedidos pelos caracteres 0x (zero e a letra x). O comando SQL

select 0x53 & 0x51 & 0x4C **as** x

produz

x
SQL

Note que cada literal corresponde a um caractere. Assim, a concatenação dos três caracteres resultou na string “SQL”.

∴

Literais hexadecimais podem também ser utilizados como expressões. O comando

select * **from** aluno **where** nome > 0x53

produz a lista de alunos cujos nomes são maiores que a string ‘S’. Isto ocorre porque a sequência hexadecimal 53 é convertida para caracteres (a letra S) por ocasião da comparação com o atributo *nome*.



No Access, cada literal hexadecimal corresponde a um caractere somente.

OPERADORES ARITMÉTICOS NO ACCESS

Operador aritmético	Operação
+	soma
-	subtração
*	multiplicação
/	divisão real
\	divisão inteira
mod	resto da divisão
^	exponenciação

O operador aritmético \ efetua a divisão inteira (resultado sem decimais) de dois números inteiros: 15 div 4 tem por resultado 3. O operador **mod** produz o resto da divisão de dois números: 10 mod 4 é igual a 2. O operador ^ implementa a exponenciação: $5^3 = 125$. O segundo operando deve ser sempre um inteiro.

OPERADORES DE COMPARAÇÃO NO ACCESS

<i>Operador de comparação</i>	<i>Significado</i>
=	igual a
<>	diferente de
>	maior que
<	menor que
>=	maior que ou igual a
<=	menor que ou igual a

OPERADORES LÓGICOS NO ACCESS

<i>Operador lógico</i>	<i>Significado</i>
and	conjunção lógica
or	disjunção lógica
not	negação lógica
xor	ou exclusivo
imp	implicação lógica
eqv	equivalência lógica

O operador **xor** implementa o *ou lógico*, cuja equivalência lógica é

$$x \text{ xor } y = (x \text{ and } (\text{not } y)) \text{ or } ((\text{not } x) \text{ and } y)$$

O operador *xor* não depende da ordem dos operadores, uma vez que

$$x \text{ xor } y = y \text{ xor } x$$

Desse modo, a tabela abaixo, que mostra as possíveis combinações de operandos, pode ser lida com os operandos na ordem inversa.

V xor V	→ F
V xor F	→ V
V xor ?	→ ?
F xor F	→ F
F xor ?	→ ?

O operador **imp** implementa a implicação lógica, $x \rightarrow y$, e sua expressão lógica é

$$x \rightarrow y = (\text{not } x) \text{ or } y$$

A tabela a seguir mostra o resultado deste operador, de acordo com seus operandos, na ordem.

V imp V	→ V
V imp F	→ F

$V \text{ imp } ? \rightarrow ?$
 $F \text{ imp } V \rightarrow V$
 $F \text{ imp } F \rightarrow V$
 $F \text{ imp } ? \rightarrow V$
 $? \text{ imp } V \rightarrow V$
 $? \text{ imp } F \rightarrow ?$
 $? \text{ imp } ? \rightarrow ?$

O operador **eqv** implementa a equivalência lógica, $x \leftrightarrow y$ e sua expressão lógica é

$$x \leftrightarrow y \equiv ((\text{not } x) \text{ or } y) \text{ and } ((\text{not } y) \text{ or } x)$$

Intuitivamente, o resultado do operador *eqv* é verdadeiro quando seus dois operandos têm o mesmo valor lógico. A tabela a seguir mostra o resultado do operador *eqv* de acordo com seus operandos, sendo a ordem irrelevante.

$V \text{ eqv } V \rightarrow V$
 $V \text{ eqv } F \rightarrow F$
 $V \text{ eqv } ? \rightarrow ?$
 $F \text{ eqv } F \rightarrow V$
 $F \text{ eqv } ? \rightarrow ?$
 $? \text{ eqv } ? \rightarrow ?$

∴



No Access, a comparação de caracteres normalmente não é sensível a maiúsculas e minúsculas.

PADRÕES DE COMPARAÇÃO NO ACCESS

<i>Caracter curinga</i>	<i>Casa com ...</i>
*	<i>qualquer seqüência de zero ou mais caracteres</i>
?	<i>qualquer caracter</i>
#	<i>Qualquer dígito numérico [0-9]</i>
[ListaDeCaracteres]	<i>qualquer caracter contido na lista de caracteres</i>
[!ListaDeCaracteres]	<i>qualquer caracter não contido na lista de caracteres</i>

ListaDeCaracteres representa um conjunto de caracteres, formado por caracteres simples ou por intervalos de caracteres. Assim, [xyz] representa os caracteres xyz; [a-m] representa os caracteres de *a* até *m*; [a-z0-9@] representa os caracteres de *a* até *z*, mais os caracteres numéricos de 0 até 9, mais o caracter @.

Vejamos alguns exemplos da aplicação de padrões de comparação no Access.

<i>Operador com padrão</i>	<i>Significado</i>	<i>Resultado</i>
nome like '[ae]*'	<i>O segundo caracter deve ser igual à vogal a ou à vogal e</i>	Carlos Azambuja Marina Azambuja Pedro Amarante Zenubio Siqueira Lenira Rocha
nome like '[A-M]*'	<i>Nomes que começam com letras de</i>	Carlos Azambuja

	<i>A até M</i>	Marina Azambuja Lenira Rocha
nome like '[!A-M]*'	<i>Nomes que começam com letras fora do intervalo de A até M</i>	Olivia Straw Pedro Amarante Zenubio Siqueira Silvia Ferreira
nome not like '[A-M]*'	<i>Nomes que não começam com letras de A até M</i>	Olivia Straw Pedro Amarante Zenubio Siqueira Silvia Ferreira
nome like '[AEIOU]*'	<i>Nomes que começam com vogais</i>	Olivia Straw
nome like '[!AEIOU]*[aeiou]'	<i>Nomes que começam com consoantes (não vogais) e terminam com vogais</i>	Carlos Azambuja Marina Azambuja Pedro Amarante Zenubio Siqueira Lenira Rocha Silvia Ferreira

O Access não utiliza um caracter de escape explicitamente, mas um caracter curinga pode ser utilizado literalmente num padrão se escrito entre colchetes. O padrão `[*]` casa com o próprio caracter asterisco. Os caracteres `*?#[` (asterisco, interrogação, jogo da velha, exclamação, abre colchete) devem aparecer entre colchetes para perder seus poderes de curinga. O caracter `]` (fecha colchete) somente é curinga quando aparece num grupo, ou seja, quando tiver sido precedido por um abre colchetes. O caracter hífen, quando aparece no início ou final de um grupo, ou fora de um grupo, casa literalmente com o próprio caracter.

PRECEDÊNCIA DOS OPERADORES NO ACCESS

<i>Operador</i>	<i>Significado</i>	<i>Precedência</i>
<code>^</code>	exponenciação	14
<code>+</code>	sinal positivo	13
<code>-</code>	sinal negativo	
<code>*</code>	multiplicação	12
<code>/</code>	divisão	
<code>\</code>	divisão de inteiros	11
mod	resto da divisão	10
<code>+</code>	soma	9
<code>-</code>	subtração	
&	concatenação	8
<code>=</code>	igual a	7
<code><></code>	diferente de	
<code><</code>	menor que	
<code>></code>	maior que	
<code><=</code>	menor ou igual a	
<code>>=</code>	maior ou igual a	
like, not like	comparação por padrão	

between, not between	comparação de intervalo	
in, not in	comparação de lista	
not	negação lógica	6
and	conjunção lógica	5
or	disjunção lógica	4
xor	conjunção lógica exclusiva	3
equiv	equivalência lógica	2
imp	implicação lógica	1

4.15 LITERAIS E OPERADORES NO MYSQL

LITERAIS NO MYSQL

Strings

Barras invertidas têm o papel de caracteres de escape numa *string*. As seguintes seqüências são interpretadas à parte.

<code>\0</code>	Caractere ASCII de código 0
<code>\b</code>	Espaço em branco
<code>\n</code>	Caractere <i>linefeed</i>
<code>\r</code>	Caracteres <i>carriage return</i>
<code>\t</code>	Caractere <i>tab</i>
<code>\z</code>	^z (fim de arquivo)
<code>\\</code>	Caractere \ (barra invertida)
<code>\%</code>	Caractere % (percentual)
<code>_</code>	Caractere _ (sublinhado)
<code>\'</code>	Caractere ' (aspas simples)
<code>\"</code>	Caractere " (aspas duplas)

Veja alguns exemplos de literais juntamente com o valor real de cada um.

<i>Literal codificado</i>	<i>Valor interpretado</i>
<code>'ABC\bDEF'</code>	ABC DEF
<code>'ABC'DEF'</code>	ABC'DEF
<code>'ABC\\bDEF'</code>	ABC\bDEF

Datas e horários

O ano de uma data pode ser representado por até quatro caracteres numéricos. Anos entre 0 e 69 serão considerados como anos entre 2000 e 2069; entre 70 e 99 serão considerados entre 1970 e 1999.



Especial atenção quando uma *string* que representa uma data for convertida para um número. No exemplo abaixo, as expressões retornam valores diferentes.

```
min('2010-01-01 23:45:30')+1
min('10-01-01 23:45:30')+1
```

Hexadecimais

MySQL aceita seqüências de caracteres hexadecimais. Uma seqüência hexadecimal pode ser convertida para um número ou para uma string, dependendo do contexto em que é utilizada. Há dois formatos válidos. No primeiro, os caracteres hexadecimais aparecem entre aspas simples, precedidos pela letra *x*; no segundo, os caracteres hexadecimais são precedidos por 0x (zero e a letra *x*), sem aspas.

O comando abaixo produz duas colunas a partir de literais. Na primeira, aparece o valor convertido para caracteres. Na segunda, o valor hexadecimal aparece convertido para a base decimal, uma vez que o literal foi empregado numa expressão aritmética.

```
select 0x53514C x, 0x53514C +1 -1 y
```

produz

x	y
SQL	5460300

O mesmo efeito poderia ter sido obtido pelo comando

```
select x'53514C' x, x'53514C' +1 -1 y
```

∴

Valores hexadecimais podem aparecer como expressões num comando SQL, como abaixo.

```
select * from aluno where nome > 0x53
```

Neste caso, o resultado contém os alunos cujos nomes são maiores que a string 'S'. Isto ocorre porque a seqüência hexadecimal 53 é convertida para caracteres (a letra S) por ocasião da comparação com o atributo *nome*. Na nossa base de exemplos, o resultado seria

matricula	nome	sexo	codcurso	nascimento
1008	Sacadura Miranda	M	INF	12/12/81

OPERADORES ARITMÉTICOS NO MYSQL

Operador aritmético	Operação
+	soma
-	subtração
*	multiplicação
/	divisão real
div	divisão inteira
%	resto da divisão
mod	resto da divisão

<<	deslocamento de bits para a esquerda
>>	deslocamento de bits para a direita
	ou lógico, sobre bits
&	and lógico, sobre bits
^	ou exclusivo lógico, sobre bits
~	negação (complemento a um), sobre bits



A divisão por zero tem como resultado o valor nulo.

Os operadores sobre bits operam sobre os tipos inteiros, *binary* e *varbinary*, e realizam as operações lógicas sobre cada um dos bits de seus operandos. Cada bit do resultado advém da aplicação do operador sobre cada par de bits dos operandos nas posições correspondentes. Os operandos devem ter o mesmo número de bits ou devem ser convertidos para algum tipo comum, se necessário. Tomando-se por base os operandos Op1 e Op2, com valores

```
Op1 = 10000110
Op2 = 01010111
```

as seguintes expressões têm por valor

```
Op1 | Op2      = 11010111
Op1 & Op2      = 00000110
Op1 ^ Op2      = 11010001
~ Op1          = 01111001
~ Op2          = 10101000
Op1 >> 2       = 00100001
Op2 << 1       = 10101110
```

Normalmente, a conversão para um tipo comum, com o mesmo número de bits, é feita automaticamente por ocasião da aplicação do operador.

OPERADORES DE COMPARAÇÃO NO MY SQL

<i>Operador de comparação</i>	<i>Significado</i>
=	igual a
<>	diferente de
!=	diferente de
>	maior que
<	menor que
>=	maior que ou igual a
<=	menor que ou igual a
<=>	igualdade com suporte a valores nulos (<i>vide seção sobre nulos mais adiante</i>)

No MySQL, as comparações com strings não distinguem maiúsculas e minúsculas. Para levar em conta essa distinção é preciso usar a palavra-chave **binary** antes de qualquer dos operandos da comparação. Exemplos:

```
'A' = 'a'          V
binary 'A' = 'a'   F
binary 'A' = binary 'a' F
```

<code>'A' = binary 'a'</code>	F
<code>binary 'A' < 'a'</code>	V
<code>'A' < 'a'</code>	F

OPERADORES LÓGICOS NO MySQL

Dos operadores lógicos, três são sinônimos dos já conhecidos *not*, *and* e *or*. O operador **xor** já foi apresentado neste capítulo.

<i>Operador lógico</i>	<i>Significado</i>
and	conjunção lógica
or	disjunção lógica
not	negação lógica
!	negação lógica
&&	equivalente à conjunção lógica
 	equivalente à disjunção lógica
xor	disjunção lógica exclusiva

Um exemplo de utilização da sintaxe MySQL de operadores lógicos pode ser observado com a consulta



```
select nome, codcurso, nascimento
from aluno
where !( nascimento < '01/01/1979' && codcurso = 'JOR' )
```

PADRÕES DE COMPARAÇÃO NO MYSQL

MySQL aceita padrões de comparação sob duas formas. Na primeira, com o operador **like**, valem os seguintes caracteres curingas:

<i>Caracter curinga</i>	<i>Casa com ...</i>
<code>%</code>	<i>qualquer seqüência de zero ou mais caracteres</i>
<code>_</code>	<i>qualquer caracter</i>

Com este operador, normalmente o caracter “\” é o caracter de escape. Sendo necessário, é possível designar outro caracter de escape, através da cláusula **escape**. A consulta

```
select nome
from professor
where nome like '%?_%' escape '?'
```

produz os nomes de professores que contêm o caracter “_”. Como este caracter é curinga, é preciso desligar essa função para que o mesmo seja usado literalmente nas comparações. Isso é feito na própria sintaxe do operador *like*, que permite a definição do caracter de escape.

A segunda forma aparece com o operador **regexp**⁸, que é uma variação do operador **like**, porém com uma gama maior de caracteres curingas. O operador *regexp* permite a construção de pa-

⁸ O operador **rlike** é sinônimo de **regexp**.

drões de comparação através de expressões regulares em conformidade com o POSIX 1003.2⁹ e constitui um poderoso recurso para busca em campos de texto.

<i>Caracter curinga</i>	<i>Casa com ...</i>
<code>^</code>	<i>início da string</i>
<code>\$</code>	<i>fim da string</i>
<code>.</code>	<i>qualquer caracter</i>
<code>x*</code>	<i>qualquer seqüência de zero ou mais caracteres x</i>
<code>x+</code>	<i>qualquer seqüência de um ou mais caracteres x</i>
<code>x?</code>	<i>zero ou um caracter x</i>
<code>xyz abc</code>	<i>os caracteres xyz ou os caracteres abc</i>
<code>(xyz)*</code>	<i>zero ou mais strings com os caracteres xyz</i>
<code>[ListaDeCaracteres]</code>	<i>qualquer caracter contido na lista de caracteres</i>
<code>[^ListaDeCaracteres]</code>	<i>qualquer caracter não contido na lista de caracteres</i>
<code>[.seqüência.]</code>	<i>exatamente a seqüência de caracteres entre os delimitadores [. e .]</i>
<code>[=CaracterDaClasse=]</code>	<i>qualquer caracter pertencente à classe de caracteres à qual o caracter de classe pertence</i>
<code>[:ClasseDeCaracteres:]</code>	<i>qualquer caracter pertencente à classe de caracteres referenciada</i>
<code>[[<:]]</code>	<i>seqüência nula no início de uma palavra; uma palavra é uma seqüência de caracteres alfanuméricos não precedida ou seguida de caracteres alfanuméricos</i>
<code>[[>:]]</code>	<i>seqüência nula no final de uma palavra</i>

Os caracteres `*` + `?` (asterisco, sinal positivo, interrogação) podem ser generalizados através da utilização de chaves com a indicação do número de repetições do padrão. Assim, `a*` é equivalente a `a{0,}`; `a+` é equivalente a `a{1,}` e `a?` é equivalente a `a{0,1}`. Se apenas um número *m* aparece entre as chaves, considera-se exatamente *m* repetições do padrão; se *m* aparece seguido de uma vírgula, considera-se no mínimo *m* repetições, com limite superior em aberto; se *m* aparece seguido de uma vírgula e um número *n*, considera-se no mínimo *m* e no máximo *n* repetições, sendo $m \leq n$.

As seguintes classes de caracteres são predefinidas no MySQL.

<code>alnum</code>	<i>caracteres alfabéticos e numéricos, incluindo A-Z, a-z, 0-9</i>
<code>alfa</code>	<i>caracteres alfabéticos, incluindo A-Z, a-z</i>
<code>blank</code>	<i>espaço ou tab</i>
<code>cntrl</code>	<i>caracteres de controle, incluindo ^c, ^x, etc.</i>
<code>digit</code>	<i>caracteres numéricos, incluindo 0-9</i>
<code>graph</code>	<i>caracteres excluindo espaços e caracteres de controle</i>
<code>lower</code>	<i>caracteres alfabéticos minúsculos, incluindo a-z</i>
<code>print</code>	<i>caracteres excluindo caracteres de controle</i>
<code>punct</code>	<i>caracteres de pontuação, incluindo !?,: etc</i>
<code>space</code>	<i>caracteres impressos em branco, incluindo espaço, line-feed, etc</i>

⁹ Padrão para codificação de expressões regulares.

upper	caracteres alfabéticos maiúsculos, incluindo A-Z
xdigit	caracteres hexadecimais, incluindo 0-9, A-F, a-f

A tabela a seguir mostra alguns exemplos de aplicação desses padrões.

Operador com padrão	Significado	Resultado
nome like '_a%' or nome like '_e%'	O segundo caracter deve ser igual à vogal a ou à vogal e	Carlos Azambuja Marina Azambuja Pedro Amarante Zenubio Siqueira Lenira Rocha
nome regexp 'Azambuja\$'	Nomes que terminam com A-zambuja	Carlos Azambuja Marina Azambuja
nome regexp '^.[ae]*'	O segundo caracter deve ser igual à vogal a ou à vogal e	Carlos Azambuja Marina Azambuja Pedro Amarante Zenubio Siqueira Lenira Rocha
nome regexp '^[A-M]'	Nomes que começam com letras de A até M	Carlos Azambuja Marina Azambuja Lenira Rocha
nome regexp '^^[^A-M]'	Nomes que começam com letras fora do intervalo de A até M	Olivia Straw Pedro Amarante Zenubio Siqueira Sílvia Ferreira
nome not regexp '^[A-M]'	Nomes que não começam com letras de A até M	Olivia Straw Pedro Amarante Zenubio Siqueira Sílvia Ferreira
nome regexp '[aeiou]*'	Nomes que começam com vogais	Olivia Straw
nome regexp '^[^AEIOU].*[aeiou]\$'	Nomes que começam com consoantes (não vogais) e terminam com vogais	Carlos Azambuja Marina Azambuja Pedro Amarante Zenubio Siqueira Lenira Rocha Sílvia Ferreira
nome regexp '([^aeiou][aeiou]){3,}'	Nomes que contém pelo menos três seqüências de não vogal seguida de vogal	Marina Azambuja Pedro Amarante Zenubio Siqueira Lenira Rocha
nome regexp binary '([^aeiou][aeiou]){3,}'	Nomes que contém pelo menos três seqüências seguidas de não vogal seguida de vogal (as vogais devem ser minúsculas)	Marina Azambuja Zenubio Siqueira Lenira Rocha



Uma expressão lógica de comparação com o operador *regexp* é verdadeira se o padrão definido casa com qualquer trecho da string de comparação. Isto é diferente do uso dos padrões no operador *like*, que requer que o padrão case com toda a string sendo testada.

A consulta que retorna os nomes dos professores que têm Azambuja como último sobrenome mostra claramente isso.

```
select nome
from professor
```

```
where nome regexp 'Azambuja$'
```

Como é suficiente que o padrão case com algum trecho do nome, não é preciso especificar o padrão que casa com a substring que antecede o sobrenome Azambuja. No caso do operador *like*, seria necessário completar o padrão, o que é feito pelo caracter % na consulta a seguir.

```
select nome
from professor
where nome like '%Azambuja'
```

PRECEDÊNCIA DOS OPERADORES NO MYSQL

<i>Operador</i>	<i>Significado</i>	<i>Precedência</i>
+	sinal positivo	11
-	sinal negativo	
not	negação lógica	10
!	negação lógica	
*	multiplicação	9
div	divisão inteira	
/	divisão real	
mod	resto da divisão	
%	resto da divisão	
+	soma	8
-	subtração	
<<	deslocamento de bits para a esquerda	7
>>	deslocamento de bits para a direita	
	ou lógico, sobre bits	6
&	and lógico, sobre bits	
^	ou exclusivo lógico, sobre bits	
~	negação (complemento a um), sobre bits	5
=	igual a	4
<>	diferente de	
!=	diferente de	
>	maior que	
<	menor que	
>=	maior que ou igual a	
<=	menor que ou igual a	
<=>	igualdade com suporte a valores nulos	
like, not like	comparação por padrão	
rlike, not rlike	comparação por padrão	
regexp, not regexp	comparação por padrão	
between, not between	comparação de intervalo	

in, not in	comparação de lista	
xor	disjunção lógica exclusiva	3
and	conjunção lógica	2
&&	conjunção lógica	
or	disjunção lógica	1
 	disjunção lógica	

4.16 LITERAIS E OPERADORES NO ORACLE

LITERAIS NO ORACLE

Números

Os caracteres usados como ponto decimal e separador de milhares podem ser alterados. O comando

```
set nls_numeric_characters = "<ponto-decimal><separador de milhares>"
```

Determina como os literais de números inteiros e reais devem ser escritos. Normalmente, são adotados os caracteres . (ponto) e , (vírgula). Um comando como

```
set nls_numeric_characters = ",."
```

Faria com que a vírgula passasse a fazer o papel de ponto decimal (como usamos no Brasil) e o ponto seria o separador de milhares.

Datas e horários

Oracle aceita uma quantidade grande de formatos para datas, que podem especificados através da diretiva **nls_date_format** ou através da função *ToDate*.

O comando **alter session** estabelece o formato para datas, com a seguinte sintaxe básica.

```
alter session set nls_date_format = <formato>
```

O formato assim estabelecido vale tanto para *strings* de “entrada”, em comandos SQL, como para resultados apresentados pelo Oracle.

A construção de um formato baseia-se na definição 1) da apresentação dos componentes de uma data, como o dia, mês, ano, hora, etc, 2) da ordem de apresentação e 3) dos caracteres separadores que poderão ser usados. Os comandos abaixo ilustram alguns exemplos.

```
alter session set nls_date_format = 'dd/mm/yyyy';
select * from aluno where nascimento > '31/12/1980'

alter session set nls_date_format = 'mm/dd/yyyy';
select * from aluno where nascimento > '12/31/1980'

alter session set nls_date_format = 'yyyy-mm-dd';
select * from aluno where nascimento > '1980-12-31'

alter session set nls_date_format = 'dd:mm:yyyy';
select * from aluno where nascimento > '31/12/1980'

alter session set nls_date_format = 'dd:month:yyyy';
select * from aluno where nascimento > '31:dezembro:1980'

alter session set nls_date_format = 'ddd.yyyy';
select * from aluno where nascimento > '365.1980'
```

```
alter session set nls_date_format = 'dd/mm/rr';
select * from aluno where nascimento > '31/12/1980'
```

No primeiro exemplo, a ordem dos componentes é dia, mês, ano, usando a barra como separador; o segundo exemplo é semelhante, apenas com a ordem invertida, no estilo americano. No terceiro exemplo, o formato é idêntico ao usado no MySQL, com ano, mês e dia separados pelo caractere – (hifen), nesta ordem. No quarto exemplo, embora o separador escolhido tenha sido o caractere : (dois pontos), a data no comando pode aparecer separada por barras. Neste caso, entretanto, o resultado do comando select aparece de acordo com o formato inicialmente escolhido. O quinto exemplo mostra uma variação interessante, onde o mês pode ser identificado pelo nome, literalmente (note que esta opção depende do idioma vigente). O sexto exemplo identifica o dia com referência ao ano – o 365º dia do ano é 31 de dezembro.

Algumas das principais opções para formatar datas são:

<i>Símbolo</i>	<i>Interpretação</i>	<i>Exemplos</i>	<i>String</i>
- / , . :	Utilizados como pontuação ou separadores	dd:mm:yyyy	31:12:2010
yyyy	Ano com 4 dígitos. Também yy, yy, y	dd:mm:yyyy	31:12:2010
dd	Dia do mês (numérico)	mm/dd/yyyy	12/31/2010
mm	Mês do ano	mm:dd:yyy	12/31/010
ddd	Dia do ano (numérico)	ddd/yyyy	365/2010
hh	Hora do dia (12 horas)	dd/mm/yyyy hh	31/12/2010 12
mi	Minutos	dd/mm/yy/hh/mi	31/12/10/12/30
ss	Segundos	dd/mm/yy hh:mi:ss	31/12/10 12:30:12
ff	Fração de segundos	dd/mm/yy hh:mi:ss:ff	31/12/10 12:30:12:10
month	Nome do mês por extenso	dd/month/yyyy	31/Dezembro/2010
mon	Nome do mês abreviado	dd:mon:yyyy	31:Dez:2010
rm	Mês do ano em números romanos	dd/rm/yyyy	31/xii/2010
rr	Transformação de anos com 2 dígitos	dd/mm/rr	31/12/10
rrrr	O mesmo que rr, porém aceita anos com 2 ou 4 dígitos	dd/mm/rrrr	31/12/2010
j	Calendário Juliano (dias desde 01/01/-4712)	j	1000
sssss	Número de segundos após a meia-noite	dd/mm/yyyy:sssss	31/12/2010:20000

Os símbolos *rr* e *rrrr* determinam a interpretação de anos com dois dígitos. Uma data com ano de dois dígitos, menor que 50, na segunda metade do século corrente, é considerada uma data do século seguinte; se o ano de dois dígitos for maior ou igual a 50, na primeira metade do século, a data é considerada do século anterior. Caso contrário, a data é sempre do século corrente.

∴

O exemplo mostra uma data escrita com o mês em algarismos romanos, usando o formato *rm*.

```
select to_date('31/xii/2010','dd/rm/yyyy') as x
from dual
```



x
31/12/2010

Outro exemplo, agora no formato *j*, que especifica o número de dias após o ano -4712!

```
select to_date(2455562,'j') as x
from dual
```



x
31/12/2010

No próximo exemplo, o comando produz uma data completa, 20.000 segundos depois da meia-noite. Note que, neste caso, o formato de datas da sessão foi alterado, de modo que a data no resultado foi apresentada de forma completa, com dia, hora, minutos e segundos.

```
alter session set nls_date_format = 'dd/mm/yyyy hh:mi:ss';
select to_date('31/12/2010:20000','dd/mm/yyyy:sssss') as x
from dual;
```



x
31/12/2010 05:33:20

∴

Oracle trabalha com um tipo de literal chamado intervalo de tempo (*interval*), que armazena um intervalo de tempo que pode ser um número de anos e meses (**month to year**) ou um número de dias, horas, minutos e segundos (**day to second**). O formato básico para a especificação de intervalo é

<expressão de intervalo> { **day to second** | **month to year** }

O exemplo abaixo lista, para cada aluno, quantos anos e meses após o ano de 2.000 o mesmo completará 10.000 dias de idade.

```
select ((nascimento + 10000) - to_date('01/01/2000')) year to month as x
from aluno
```



x
+000000007-06
+000000006-03
+000000007-00
+000000007-03
+000000004-10
+000000009-04
+000000003-00

Um outro exemplo de uso de intervalo é a consulta que retorna o número de dias, horas, minutos e segundos decorridos desde o início do dia corrente. Note que a função *sysdate* retorna a data completa, com horário. A expressão *to_date(sysdate,'dd/mm/yyyy')* retorna o mesmo valor, porém truncado, por força do formato utilizado na conversão (sem escrever horas, minutos e segundos).¹⁰

```
select (sysdate - to_date(sysdate,'dd/mm/yyyy')) day to second as x
from dual
```



x
+000000000 18:48:02

¹⁰ **dual** é uma fonte de registros virtual, referenciável no Oracle, tratada como uma tabela de apenas um registro.

Supondo-se que o comando tenha sido executado às 18h 48min e 2 segundos, em qualquer data, o resultado é o que é mostrado acima.

Hexadecimais

Oracle não aceita a codificação de literais hexadecimais no corpo de comandos SQL. Entretanto, alguns utilitários, como o SQL*Loader, aceitam.

OPERADORES ARITMÉTICOS NO ORACLE

<i>Operador aritmético</i>	<i>Operação</i>
+	soma
-	subtração
*	multiplicação
/	divisão real

OPERADORES DE COMPARAÇÃO NO ORACLE

<i>Operador de comparação</i>	<i>Significado</i>
=	igual a
<>	diferente de
!=	diferente de
^=	diferente de
≠	diferente de
>	maior que
<	menor que
>=	maior que ou igual a
<=	menor que ou igual a

Na comparação de *strings*, o Oracle sempre leva em consideração maiúsculas e minúsculas. Se o programador desejar, pode anular a sensibilidade pelo uso de funções de conversão como **upper**, apresentada no capítulo 8, que padroniza uma string com caracteres maiúsculos.

OPERADORES LÓGICOS NO ORACLE

<i>Operador lógico</i>	<i>Significado</i>
and	conjunção lógica
or	disjunção lógica
not	negação lógica

PADRÕES DE COMPARAÇÃO NO ORACLE

No Oracle, a codificação de padrões de comparação pode ser feita a partir de dois caracteres curingas, como mostra o quadro abaixo.

<i>Caracter curinga</i>	<i>Casa com ...</i>
%	<i>qualquer sequência de zero ou mais caracteres</i>
_	<i>qualquer caracter</i>



No Oracle, o operador **like** distingue maiúsculas e minúsculas.

A exemplo do MySQL, o caracter de escape, quando necessário, é definido no próprio comando SQL, através da cláusula **escape**, como na consulta

```
select nome
from professor
where nome like '%\_%' escape '\'
```

que produz os nomes de professores que contêm o caracter “_”.

PRECEDÊNCIA DOS OPERADORES NO ORACLE

<i>Operador</i>	<i>Significado</i>	<i>Precedência</i>
+	sinal positivo	7
-	sinal negativo	
*	multiplicação	6
/	divisão	
+	soma	5
-	subtração	
	concatenação	
=	igual a	4
!=	diferente de	
<>	diferente de	
≠	diferente de	
<	menor que	
>	maior que	
<=	menor ou igual a	
>=	maior ou igual a	
like, not like	comparação por padrão	
between, not between	comparação de intervalo	
in, not in	comparação de lista	3
not	negação lógica	
and	conjunção lógica	
or	disjunção lógica	1

4.17 LITERAIS E OPERADORES NO SQL SERVER

LITERAIS NO SQL SERVER

Datas e horários

No SQL Server é possível configurar a ordem dos componentes de uma data de várias formas, bem como é possível usar nomes escritos por extenso ou abreviadamente. Supondo-se que a ordem estabelecida seja mês/dia/ano e que o idioma padrão seja o inglês, os seguintes literais são válidos.

```
'Jan 31, 2010'
'January 31 2010'
'01/31/2010'
```



Normalmente, a ordem que prevalece para *scripts* SQL é mês/dia/ano e o idioma padrão é o inglês americano. Neste livro, a maior parte dos exemplos as datas aparecem na ordem brasileira, dia/mês/ano. Para mudar, basta executar o comando

```
set dateformat mdy;
```

As variações aceitas por este comando são *mdy*, *dmy*, *ymd*, *ydm*, *myd* e *dym*.

A ordem dos componentes de uma data está associada ao idioma padrão. Nos exemplos acima, o idioma de referência é o inglês, o que determina que os meses sejam nomeados de acordo. O comando

```
set language 'Português';
```

muda o idioma padrão para o português, fazendo com que os nomes de meses possam ser escritos no idioma de Camões.

```
'31 Jan 2010'
'31 Janeiro 2010'
'31/01/2010'
```

No SQL Server, é possível escrever datas sem separadores mas, neste caso, a ordem deve ser, sempre, ano/mês/dia, independentemente do formato que estiver em uso. Ou seja, ainda que a ordem escolhida tenha sido *dmy*, o literal

```
'20100131'
```

é válido, mas o literal

```
'31012010'
```

não é.

∴

Quando o nome do mês é usado, por extenso ou de forma abreviada, a ordem do mês na *string* não é relevante. Por exemplo, se a ordem vigente for *dmy*, ambos os literais abaixo são válidos e interpretados corretamente.

```
'31 Janeiro 2010'
'Janeiro 31 2010'
```

∴

Os exemplos abaixo mostram o efeito de literais de data escritos não exatamente de acordo com as definições de idioma e formato de data em vigor. Note que o resultado das consultas é rigorosamente o mesmo.


```

set language 'Português';
set dateformat dmy;
select * from aluno where nascimento > '31 Jan 1980';
select * from aluno where nascimento > 'Jan 31 1980';
select * from aluno where nascimento > '1980 Jan 31';

set language 'us_english';
set dateformat dmy;
select * from aluno where nascimento > '31 Jan 1980';
select * from aluno where nascimento > '31 January 1980';
select * from aluno where nascimento > '19800131';

```



matricula	nome	sexo	codcurso	nascimento
1001	Ricardo Biondi	M	DIR	21/02/80
1002	Maria Rita Colatti	F	INF	10/11/78
1004	Oscarito Vianna	M	DIR	14/08/79
1005	Barbara Carlito	F	JOR	29/10/79
1007	Carlos Maradona	M	DIR	30/06/77
1008	Sacadura Miranda	M	INF	12/12/81
1010	Maria Lucia Silva	F	JOR	10/08/75

∴

Os idiomas disponíveis na instalação em uso podem ser obtidos numa consulta ao dicionário de dados, através do comando

```
select * from master.dbo.syslanguages
```

Hexadecimais

Cadeias de caracteres hexadecimais podem ser utilizadas no SQL Server, mas devem ser expressamente convertidas. O comando abaixo produz três colunas a partir de valores hexadecimais. Na primeira, aparece o valor sem conversão, na segunda o valor convertido para caracteres e na terceira, o número inteiro correspondente.

```
select 0x53514C x, convert(char(3), 0x53514C) y, convert(int, 0x53514C) z
```

produz

x	y	z
0x53514C	SQL	5460300

∴

Valores hexadecimais podem também ser utilizados como expressões. O comando

```
select * from aluno where nome > 0x53
```

produz a lista de alunos cujos nomes são maiores que a string 'S'. Isto ocorre porque a sequência hexadecimal 53 é convertida para caracteres (a letra S) por ocasião da comparação com o atributo *nome*.

OPERADORES ARITMÉTICOS NO SQL SERVER

Operador aritmético	Operação
+	soma
-	subtração
*	multiplicação
/	divisão real
%	resto da divisão
	ou lógico, sobre bits
&	and lógico, sobre bits
^	ou exclusivo lógico, sobre bits
~	negação (complemento a um), sobre bits

Os operadores sobre bits operam sobre os tipos inteiros, *binary* e *varbinary*, e realizam as operações lógicas sobre cada um dos bits de seus operandos. Exemplos da aplicação desses opera-

dores podem ser encontrados na seção sobre os operadores aritméticos do MySQL, apresentados anteriormente neste capítulo.

OPERADORES DE COMPARAÇÃO NO SQL SERVER

<i>Operador de comparação</i>	<i>Significado</i>
=	igual a
<>	diferente de
!=	diferente de
>	maior que
<	menor que
>=	maior que ou igual a
<=	menor que ou igual a
!>	não maior que
!<	não menor que

A sensibilidade a maiúsculas e minúsculas no SQL Server depende dos parâmetros que controlam a instalação. Em condições normais, os servidores SQL são instalados sem essa sensibilidade, ou seja, não há distinção entre maiúsculas e minúsculas nas comparações. No *MS Windows*, a propriedade de registro que altera essa configuração é denominada *CaseSensitive*.

OPERADORES LÓGICOS NO SQL SERVER

SQL Server disponibiliza apenas o conjunto básico de operadores lógicos.

<i>Operador lógico</i>	<i>Significado</i>
and	conjunção lógica
or	disjunção lógica
not	negação lógica

PADRÕES DE COMPARAÇÃO NO SQL SERVER

Os padrões de comparação permitidos no SQL Server são mostrados no quadro abaixo.

<i>Caracter curinga</i>	<i>Casa com ...</i>
%	qualquer seqüência de zero ou mais caracteres
_	qualquer caracter
[x-y]	qualquer caracter contido no intervalo entre os caracteres x e y
[abc]	qualquer caracter contido no conjunto de caracteres abc
[^x-y]	qualquer caracter não contido no intervalo entre os caracteres x e y
[^abc]	qualquer caracter não contido no conjunto de caracteres abc

O quadro a seguir mostra alguns exemplos da aplicação dos caracteres curingas acima em conjunto com o operador *like*, considerando-se os nomes da tabela PROFESSOR.

<i>Operador com padrão</i>	<i>Significado</i>	<i>Resultado</i>
nome like '_[ae]%'	<i>O segundo caracter deve ser igual à vogal a ou à vogal e</i>	Carlos Azambuja Marina Azambuja Pedro Amarante Zenubio Siqueira Lenira Rocha
nome like '[A-M]%'	<i>Nomes que começam com letras de A até M</i>	Carlos Azambuja Marina Azambuja Lenira Rocha
nome like '[^A-M]%'	<i>Nomes que começam com letras fora do intervalo de A até M</i>	Olivia Straw Pedro Amarante Zenubio Siqueira Silvia Ferreira
nome not like '[A-M]%'	<i>Nomes que não começam com letras de A até M</i>	Olivia Straw Pedro Amarante Zenubio Siqueira Silvia Ferreira
nome like '[AEIOU]%'	<i>Nomes que começam com vogais</i>	Olivia Straw
nome like '[^AEIOU]%'	<i>Nomes que começam com consoantes (não vogais) e terminam com vogais</i>	Carlos Azambuja Marina Azambuja Pedro Amarante Zenubio Siqueira Lenira Rocha Silvia Ferreira

A exemplo do MySQL e do Oracle, o caracter de escape pode ser definido no próprio comando SQL, através da cláusula **escape**. A consulta

```
select nome
from professor
where nome like '%_%' escape '\'
```

produz os nomes de professores que contêm o caracter “_”.

PRECEDÊNCIA DOS OPERADORES NO SQL SERVER

<i>Operador</i>	<i>Significado</i>	<i>Precedência</i>
+	sinal positivo	8
-	sinal negativo	
~	negação (complemento a um) sobre bits	
*	multiplicação	7
/	divisão	
%	resto da divisão	
+	soma	6
+	concatenação	

-	subtração	
=	igual a	
>	maior que	
<	menor que	
>=	maior ou igual a	5
<=	menor ou igual a	
<>	diferente de	
!=	diferente de	
!>	menor ou igual a	
!<	maior ou igual a	
^	ou exclusivo lógico sobre bits	
&	conjunção lógica sobre bits	4
	disjunção lógica sobre bits	
not	negação lógica	3
and	conjunção lógica	2
between, not between	comparação de intervalo	
in, not in	comparação de lista	1
like, not like	comparação por padrão	
or	disjunção lógica	

EXERCÍCIOS

4.1 Cada uma das expressões abaixo foi escrita de acordo com a sintaxe da implementação indicada. Calcule o valor final de cada uma, de acordo com as regras de precedência aplicáveis.



- a) $5 = 20 \bmod 15 \text{ and } 3+2 > 4$
 b) `false between (2+3 > 4 xor 2=2) and (40 \ 32 >1)`



- c) $\sim 1 | 1$
 d) `not ( 'abcdefg' regexp '^[abcdef]${})`



- e) `'x' like '% ' || 'x'`
 f) $-2 + +7 - -2$



- g) $((4 \& 7) = (4 \wedge 2)) \text{ or } 1=2$
 h) $\sim \sim 2=2$

4.2 Para cada um dos padrões de comparação abaixo, escritos na sintaxe das implementações indicadas, forneça uma versão equivalente na sintaxe das expressões regulares do MySQL.

-  a) `[x]*`
 b) `#[^0-9]###[^0-5]`
 c) `%x%`
 d) `_a%a_`
 e) `[a][a][a][a][a]%`
 f) `[abc][abc][abc]_`

4.3 Para a base exemplo, escreva uma consulta SQL que produza a lista de alunos que possuem três nomes, em ordem alfabética, como abaixo.

matricula	nome
1002	Maria Rita Colatti
1010	Maria Lucia Silva

Na solução, considere que entre um nome e outro:

- haja, sempre, apenas um caracter em branco;
- possa haver mais de um caracter em branco.

4.4 Para a base de exemplos, indique o resultado final da consulta

```
select 2+4 as numero
from aluno
```

4.5 Para a base de exemplos, escreva uma consulta SQL que mostre, para cada disciplina, sua carga horária principal e secundária, como no resultado abaixo.

natureza	disciplina	ch_principal	ch_secundaria
Prática	Compiladores	4	2
Prática	Estatística	2	2
Teórica	Bancos de Dados	4	2
Teórica	Dir. Constitucional	4	0
Teórica	Direito Civil	4	2
Teórica	Português	4	0
Teórica	Sociologia	3	1

A carga horária principal é igual a *CHST* se $CHST > CHSP$. Caso contrário, a carga principal é igual a *CHSP*. A lista deve conter ainda uma observação sobre a natureza da disciplina, de acordo com sua carga horária principal. A lista deve estar ordenada por *natureza/nome da disciplina*.

4.6 Mostre o resultado produzido pelas seguintes expressões aritméticas nas implementações onde as mesmas são aplicáveis:

```
10 div 3
1 div 0.2
1.6 div 0.5
1.4 div 0.5
```

4.7 Para a consulta abaixo, indique quais os nomes das colunas nas diversas implementações SQL:

```
select disciplina,
      chst,
      chsp,
      chst + (chsp * 0.5)
from disciplina
```

4.8 Suponha que a tabela PROFESSOR tenha um atributo a mais, denominado *salário*, e que seja preciso calcular os valores de uma folha de pagamentos incluindo os descontos do imposto de renda e os salários líquidos. As regras para cálculo do imposto são ilustradas na tabela abaixo.

<i>Faixa de renda</i>	<i>Taxa</i>	<i>Dedução</i>
até R\$ 1.164,00	0 %	R\$ 0,00
entre R\$ 1.164,01 e R\$ 2.326,00	15 %	R\$ 174,60
acima de R\$ 2.326,00	27,5 %	R\$ 465,35

Assim, o imposto de renda para um professor com salário de R\$ 3.000,00 seria calculado pela fórmula $(3.000,00 * 15 / 100) - 174,60$. Escreva uma consulta SQL que produza uma lista na forma abaixo.

professor	salario	imposto	salario liquido
...	...	...	...

